Towards Distillation Guarantees under Algorithmic Alignment: Learning Local Computations from Source Network Representations

Thien Le Melanie Weber

Abstract

Distillation transfers knowledge from a trained teacher (or source) model to a student suited to deployment. For algorithmic graph learning problems, *algorithmic alignment* motivates a graph neural network as the student architecture when its communication pattern matches that of a local algorithm for the task. We study when the teacher's internal representations make this transfer computationally efficient. Building on Boix-Adserà (2024), we introduce a *readability* condition linking linear readouts of teacher activations to the local computations of the aligned algorithm. When this condition holds exactly for rational-valued activations, and the local algorithm agrees with the teacher's predictions, we give a polynomial-time distillation algorithm with arbitrarily small error relative to the teacher's predictions. Under norm bounds, an approximate version controls how representation errors accumulate across rounds. Finally, we show a lower bound: under a standard cryptographic assumption, we construct tasks for which specified teacher activations permit efficient distillation but final predictions alone do not. Our results provide a sufficient representation condition for distillation into a graph architecture aligned with the task.

Keywords: model distillation, algorithmic alignment, graph neural networks, linear representations, local learning

1 Introduction

Distillation seeks to transfer the behavior of a trained model to a student that is easier to deploy (Hinton et al., 2015; Jiao et al., 2020). We call the trained model the *teacher*, or *source model*. The teacher may have learned from broad data with a flexible architecture, while the student can incorporate structure specific to its task, as convolutional networks do for images (Krizhevsky et al., 2012; LeCun et al., 2015; Goodfellow et al., 2016). Studies at scale examine when models can learn useful structure from data (Bachmann et al., 2023; Brehmer et al., 2025). This suggests a "learn first, distill later" approach: use the teacher's learned knowledge to train a model whose architecture reflects how the task can be solved. We study when this transfer can be made computationally efficient.

Graph problems provide a natural setting for this question. In a message-passing graph neural network (GNN), each vertex stores a vector, receives information from neighboring vertices, and updates its vector (Gilmer et al., 2017; Kipf and Welling, 2017). Many graph algorithms also compute through repeated local updates. For example, reachability within a fixed number of steps can be computed by propagating a Boolean flag from a marked vertex to its neighbors. *Algorithmic alignment* relates this match between architecture and algorithm to learnability (Xu et al., 2020). A suitable communication pattern, however, still leaves the student to learn what information to store and how to update it.

To incorporate intermediate computations, it is clear that just using the teacher's final predictions is not enough. A reachability label, for example, says whether a vertex can be reached, but not which vertices become reachable at each earlier round. Directly supervising each step would require additional labels that ordinary distillation does not provide. Moreover, the same tasks may be solved with different algorithms and circuits, using different intermediate computations. To avoid these issues, we instead allow the learner to inspect specified internal activation vectors of the teacher on sampled graphs. These vectors may contain useful information, but the learner must discover which information to use and how to combine it across the graph. Our aim is to learn this GNN from the teacher's activations and final predictions, without supervision for some reference algorithm's intermediate states. We call the final outputs *endpoint predictions* in the formal analysis, to distinguish them from the intermediate values.

Our approach builds on the representation condition (a kind of *linear representation hypothesis*) for decision-tree distillation of Boix-Adserà (2024). Suppose a local graph algorithm reproduces the teacher's predictions and uses a small decision tree for its update rule. A path in this tree specifies a sequence of tests. Each path can be associated with an indicator function that records whether all those tests are satisfied. Our assumption is that linear functions of the teacher's activations recover these indicators. We call these linear functions *readouts*, and the assumption *readability* accordingly. Note that the learner receives neither the tree nor these readouts. The condition concerns the information in the trained teacher. Under this condition, teacher representations can guide the student's choice of local features without supplying labels for the algorithm's intermediate states.

1.1 Contribution of This Paper

We give sufficient conditions for learning a local graph computation from a teacher's activations and final predictions. The learner knows the initial node values and which neighboring inputs each node may use, but learns its representations and update computations jointly. The student may use different internal states from the reference algorithm and different update rules at different rounds, as does a standard GNN. Our guarantees concern agreement with the teacher's final predictions.

- **Efficient distillation (Theorem 1).** For rational teacher activations satisfying the representation condition exactly, we learn a ReLU message-passing network in polynomial time. When the reference algorithm agrees with the teacher, the learner attains any prescribed prediction accuracy without intermediate-state labels.
- **Approximate representations (Theorem 2).** Under explicit norm bounds, we extend the learner to approximate readouts. The error bound accounts for how each round amplifies representation errors.
- **Limits of prediction-only supervision (Theorem 3).** Assuming secure pseudorandom functions, we construct tasks that are hard to learn from final-prediction queries alone but become efficiently learnable with suitable teacher activations. Both sides allow the same student class.

1.2 Related Work

Representation-guided distillation. Boix-Adserà (2024, Definition 2.2 and Theorem 3.6) develops a learning-theoretic framework for distillation and shows how readable path features support decision-tree extraction. That work also motivates using extracted features as intermediate computations in structured targets. We adopt this premise and study a different output class:

vector-state local circuits whose features and updates are learned together. Our procedure uses ordinary sampled graph evaluations. The larger circuit class is essential to the argument; this is not a proper decision-tree learning guarantee.

Learning polynomial feature spaces. The Basis Learner of Livni et al. (2014) constructs polynomial networks through products, linear-algebraic basis selection and a convex output fit. Vanishing Component Analysis likewise constructs polynomial features degree by degree (Livni et al., 2013). Related algorithms for approximately vanishing ideals use coefficient constraints and convex optimization (Wirth and Pokutta, 2022). More recently, VI-Nets combine a pretrained network prefix with a learned polynomial layer (Pelleriti et al., 2025). Our construction follows this established pattern. The source representations guide which combinations to retain, keeping the width bounded while preserving the reference tree's path features across graph rounds (Section 4).

Alignment and intermediate supervision. Xu et al. (2020) analyze learning aligned GNN updates with intermediate supervision and stability conditions under algorithmic alignment. Experimentally, FitNets trains students against intermediate teacher features (Romero et al., 2015), CLRS supplies algorithmic hints (Veličković et al., 2022), while Bevilacqua et al. (2023) study regularization in neural algorithmic reasoning. We ask when representations can replace supplied hints during distillation. Our readability assumption requires more than linear readouts of the reference algorithm's intermediate states: it also covers the tests along its decision-tree paths. This is a condition on the source model; the learner receives neither the readouts nor the intermediate states. Appendix A.2 details the comparison with intermediate supervision.

Readability and identification. Linear-representation studies motivate looking for concepts in source activations (Park et al., 2024). Empirical studies show that layer probes can recover useful predictions (Alain and Bengio, 2017; Tenney et al., 2019; Belrose et al., 2023). However, they do not imply that the source causally uses a decoded quantity (Geiger et al., 2021), that readable states have consistent local successors, or that a preferred interpretation is identifiable. Our upper bound needs one suitable reference algorithm and permits the student to choose its own vector coordinates.

2 Preliminaries

We first specify our setting. Let $G = (V, E, x, e)$ be a nonempty attributed graph with node attributes x_u and optional edge attributes e_{vu}. Fix an input distribution $\mathcal{D}$ supported on graphs with $1 \le |V| \le n$, where the integer $n \ge 1$ bounds graph size. Fix a number of computation rounds $L \ge 1$. A graph may have a distinguished root o. We write $N(u)$ for the vertices whose states are available to u in one round of message passing, usually the neighbors of u.

Local Iteration. For an iterative local algorithm, let $s^t = (s_u^t)_{u \in V}$ denote the vector of scalar node states at round t. A local algorithm has a known initialization $s^0 = \mathrm{Init}(G)$ and a shared local update F. For a specific graph G, the update acts by

$$s_u^{t+1} = F_G(s^t)_u = F\Big(s_u^t, x_u, \{(s_v^t, e_{vu}) : v \in N(u)\}\Big), \qquad 0 \le t < L. \tag{1}$$

For anonymous message passing, F is invariant to the order of neighbors. Our main results specify labeled communication ports in Section 3. Throughout the paper, we are interested in algorithmic tasks that can be solved with a reference local algorithm that uses the same Boolean update at every vertex and round (e.g. vertex reachability). The student architecture stores, for each vertex, a learned real vector and shares parameters across vertices, but may use different local modules at different rounds. This is satisfied if the student is a standard message-passing neural network.

Source Access and Distillation. We allow access to the source model's representations as well as its final predictions. A fixed deterministic source T maps each graph G to a vector

$T(G) \in \{0,1\}^{V(G)}$; write $T_u(G)$ for its prediction at vertex u. For example, a source trained for bounded-horizon reachability predicts whether each vertex is reachable from a specified starting vertex within L steps. Distillation seeks to reproduce these predictions. For simplicity, we will assume that the source model is an unstructured model organized into layers (e.g. an MLP or a transformer).

The source interface returns $T(G)$ and designated node-indexed activation vectors $Z_u^t(G) \in \mathbb{R}^{D_t}$ for target rounds $1 \le t \le L$. These $Z_u^t(G)$ may come from one layer of the source architecture or concatenate several layers. Note that target round t and source network depth are indexed differently. We fix the collection of activations that is to appear in each Z_u^t across input graphs and independently of the samples seen during distillation. For example, a Transformer may represent a graph using one input token per vertex. At each layer, the model produces one hidden vector per token, so the input token-to-vertex correspondence also indexes these hidden vectors. If a 12-layer transformer was trained to learn 3-reachability, one possible Z_u^1 could be the concatenation of token u's activations in the first 4 layers, Z_u^2 the next 4 and Z_u^3 the last 4. Different assumptions on how Z_u^t matches the t-th round computation of the reference algorithm correspond to different types of interpretability results. For instance, a *linear representation hypothesis* can ask that a linear function of Z_u^t recover the reference state s_u^t.

This source access is only used during distillation. At deployment, the student does not have access to the source model.

Error metric. We count a graph as an error if the student disagrees with the source at even one vertex. For a student $\widehat{T}$, this whole-graph *endpoint* (or *final prediction*) error is

$$\mathrm{err}_{\mathcal{D}}(\widehat{T}, T) = \Pr_{G \sim \mathcal{D}}[\exists u \in V(G) : \widehat{T}_u(G) \neq T_u(G)].$$

An (ϵ, δ) distillation guarantee makes this error at most ϵ with training confidence at least $1 - \delta$, as in Boix-Adserà (2024). For a single-decision task, such as reachability between two specified vertices, the event can be restricted to that decision.

Sampling and Notation. For Theorems 1 and 2, training graphs are independent draws from $\mathcal{D}$. Unless stated otherwise, vector norms are Euclidean. We write $(z)_+ = \max\{0, z\}$ and $\mathbf{1}\{A\}$ for an event indicator, and implement clipping by $\mathrm{clip}_{[0,1]}(z) = \mathrm{ReLU}(z) - \mathrm{ReLU}(z-1)$. The next section specifies which information in the source supports learning without intermediate labels.

3 Structured Distillation Framework

We study distillation into a message-passing neural network. We first describe this student architecture, then organize the setting and assumptions into two parts: the graph inputs and reference local algorithm, and the information that the source representations must reveal about that algorithm. Figure 1 illustrates the communication setting and the exact readability assumption.

A message-passing neural network computes through L rounds of local updates. Each vertex stores a real vector, receives vectors from its neighbors, and uses a neural network to update its vector from these messages, its own previous vector, and its attributes. We call the network used for one such update a *local update module*. All vertices share the same module within a round; different rounds may use different modules. After the final round, a shared output rule produces a prediction at each vertex.

3.1 Graph Setting and Reference-Algorithm Assumption

The learner receives the sampled graphs, source predictions, and specified activations from Section 2. The following setting details what each local update can use. Note that since we focus on algorithmic

graph tasks, there is also a reference algorithm that specifies a computation against which we compare the learned student (Figure 1). This comparison is for the analysis. Our distillation algorithm and the student network do not have direct access to this reference algorithm, but can obtain information about it from the source network. Recall that the source, its representation interface, and the reference algorithm are fixed independently of the training sample.

Known Communication and Initialization. Each graph has a Boolean node attributes and at most p labeled incoming ports at each vertex. Each port specifies one sender, usually a neighboring vertex. The learner knows this communication pattern and an efficiently computable Boolean initialization $z^0 = \mathrm{Init}(G)$. We use z^t for the Boolean reference states, specializing the states s^t of Section 2.

Figure 1(a) shows how the reference algorithm and student use the same communication pattern.

Small Local Decision Tree. For a reference-state assignment $z \in \{0,1\}^{V(G)}$, let $C_G(z)_u$ concatenate the a raw attributes at u, its own state z_u, and the states at its p labeled incoming ports, using zero for a missing port. Thus $C_G(z)_u \in \{0,1\}^q$, where $q = a + p + 1$. The learner knows this concatenation rule but does not observe the intermediate reference states used to evaluate it. We assume that one unknown Boolean decision tree $F_\star$, with at most $S \geq 1$ leaves and depth at most $r \geq 1$, generates the reference-algorithm trajectory through the shared update

$$z_u^t(G) = F_\star(C_G(z^{t-1})_u), \qquad 1 \leq t \leq L.$$

Each internal tree node tests one input bit. We call an input bit or its complement a *literal*. The same tree acts at every vertex and round, using only current states and graph inputs to compute the next states. The learner knows the complexity bounds but receives neither $F_\star$ nor the intermediate states z^t.

Note that this tree describes the reference computation and that we do not require the learner to return a tree. The student network may use wider vector states and different modules across rounds. Its learned coordinates need not recover the reference states.

3.2 Representation Assumptions

While the small-tree assumption limits the complexity of the reference computation, this is still not sufficient for efficient distillation. We also need the source representations to help the learner discover useful local computations. Following Boix-Adserà (2024), we require linear access to the tests followed along the tree's paths. A root-to-node path P in $F_\star$ defines an indicator a_P that is one precisely when all tests on that path are satisfied; Figure 1(b) highlights an example. We state two versions of the representation assumption: exact recovery for Theorem 1, and approximate recovery for Theorem 2.

Exact Path Readability. For every round t and path P, we assume there is an unknown readout vector $w_P^t \in \mathbb{R}^{D_t+1}$ such that

$$a_P(C_G(z^{t-1})_u) = (w_P^t)^\top (1, Z_u^t(G)) \quad \text{almost surely for every vertex.} \tag{2}$$

The same readout applies across graphs and vertices, but readouts may differ across paths and rounds. We fix them independently of the training sample and do not supply them to the learner. For the polynomial-time guarantee, the source returns finite-precision activations (rational values with finite binary encodings), and the equality holds for these returned values.

Approximate Path Readability. To allow imperfect recovery, let $\xi \geq 0$ bound the root mean squared (RMS) error of each path readout. We assume fixed unknown witnesses with $\|w_P^t\|_2 \leq \tau$

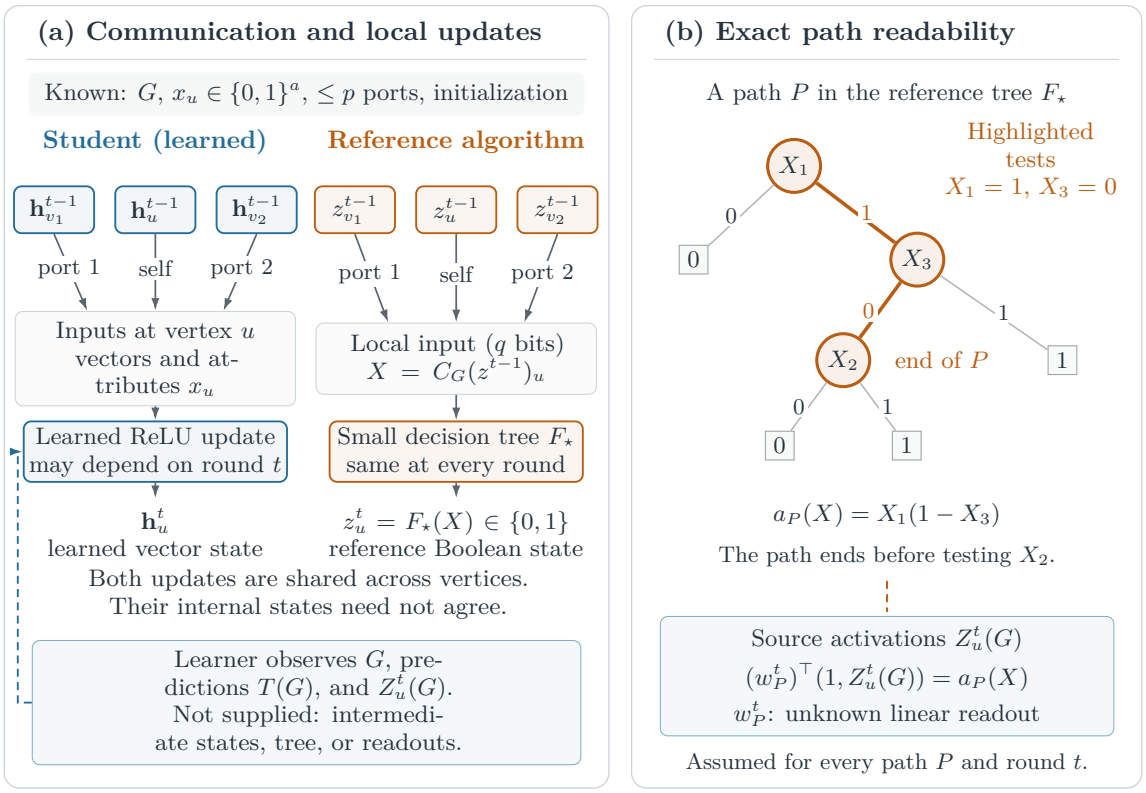


Figure 1: (a) Shared communication pattern: the student carries learned vectors $\mathbf{h}_u^t$, and the reference algorithm carries Boolean states z_u^t; v_1, v_2 denote two incoming senders. (b) Exact path readability: the highlighted tests define $a_P(X) = X_1(1 - X_3)$.

and an almost-sure feature bound $\|(1, Z_u^t(G))\|_2 \leq B_Z$, satisfying, for every t, P,

$$\mathbb{E}_{G\sim\mathcal{D}} \frac{1}{|V(G)|} \sum_u \left| a_P(C_G(z^{t-1})_u) - (w_P^t)^\top (1, Z_u^t(G)) \right|^2 \leq \xi^2. \tag{3}$$

The learner uses the bounds τ and B_Z without receiving the readouts. Note that the exact readability theorem does not require these norm bounds. The polynomial-time implementation of the approximate result also uses rational samples, as specified in Theorem 2.

Both versions require linear decoding of path indicators. Linear encoding alone does not guarantee this decoding property (Garg et al., 2026). They also require more than the state readouts illustrated in Section 2: recovering individual reference states need not reveal the conjunctions of tests on a tree path. We are now ready to state our main results.

4 Distillation Guarantees

We use the representation assumptions of Section 3 to learn the student's local computations. Under exact readability, the learner runs in polynomial time using finite precision. Under approximate readability, the same construction bounds error amplification across graph rounds.

4.1 Learning Representations from Endpoint Supervision

The learner constructs the local updates that produce the student's vector states $\mathbf{h}_u^t$ in Figure 1(a). It first builds feature functions from graph inputs, using source activations to select useful combinations, and then implements these functions with ReLU networks (Figure 2).

Overview. Draw m independent graphs $G_1, \ldots, G_m$. Each round collects a constant coordinate, raw attributes, and the previously constructed feature vectors at each vertex and its incoming ports, starting from the known initialization $(1, z^0)$. These are the student inputs on the left of Figure 1(a). Stacking the student's local input vectors gives the matrix U^t, with one row for each sampled pair (G_i, u) and one column for each input coordinate.

To construct one local update, the learner performs an inner loop that alternates between expanding its features and using source activations to retain a small set of combinations. Write $Q_{t,k}$ for the matrix of retained feature values after k such inner iterations, starting from $Q_{t,0} = \mathbf{1}$. At iteration k, the learner forms pointwise products of columns of U^t with columns of $Q_{t,k-1}$, then uses source activations to construct the next retained-feature matrix $Q_{t,k}$ (more details below). These products allow a decision-tree path to extend by one test, as illustrated by the path indicator in Figure 1(b). Only the final vectors in $Q_{t,r}$ pass to the next graph round; their ReLU approximation gives the states $\mathbf{h}_u^t$ in Figure 1(a). The learner stores the feature expressions so that it can compute them on new graphs without source access. We perform r inner iterations within each graph round because the reference tree has depth at most r.

After round L, the learner fits a linear readout to the source's final predictions. It minimizes the average, over graphs, of the largest absolute prediction error at any vertex, with a bound on the readout's coefficients. This constrained fit is a linear program. Appendix B.3 specifies the objective and constraints. After fitting, multiplication approximations on bounded inputs (Yarotsky, 2017) compile the stored expressions into a ReLU student (Appendix B.4).

Efficient fitting with source activations. The product expansion can create many features. To keep the representation small, the learner fits the D_t normalized source features as linear combinations of these products, with ridge (also called Tikhonov) regularization preventing excessive amplification of combinations that vary very little on the fitting sample. The learner then normalizes the fitted columns and clips their values to a bounded interval. The clipping threshold leaves the fitting values unchanged while controlling feature values on new graphs. The regression produces one fitted column per source feature, and together with the constant column, this gives $Q_{t,k}$ at most $D_t + 1$ columns after every inner iteration. Note that in the feature regressions and normalizations, we weight graphs equally and, within each graph, weight vertices equally. Thus larger graphs do not receive more weight in these fits. Appendix B.3 gives the details.

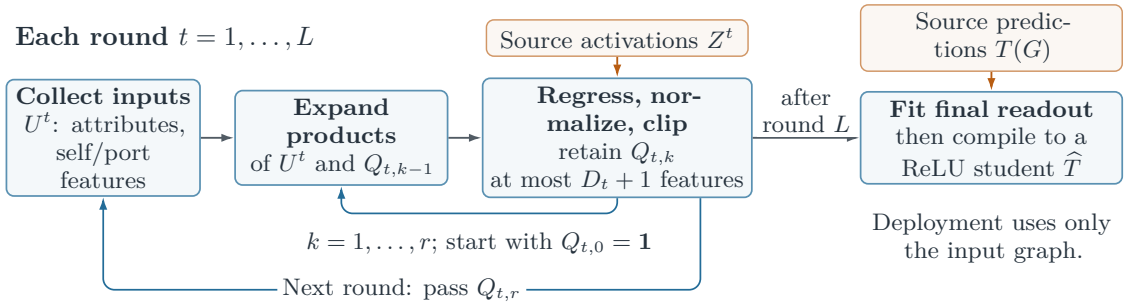


Figure 2: The distillation algorithm. At each graph round, regularized regression fits source features from the expanded products. Dashed arrows are information used during fitting.

The first theorem guarantees that exact readability makes this construction efficient. Recall that the reference algorithm need not agree perfectly with the source. We measure its final-prediction disagreement by $\rho_{\text{end}} = \Pr_{G\sim\mathcal{D}}[\exists u : z_u^L(G) \neq T_u(G)]$; when they agree almost surely, $\rho_{\text{end}} = 0$. The sample bound also uses a statistical complexity factor v for the graph losses expressible by the candidate students; the proof bounds it by a polynomial in the architecture parameters. Recall also that n bounds the number of vertices in each graph, L is the number of message-passing rounds, and r, S bound the depth and number of leaves of the reference algorithm's local decision tree, respectively. Each vertex has a Boolean attributes and at most p labeled incoming ports.

Theorem 1 (Polynomial-time distillation under exact path readability)**.** *Assume the interface of Section 3 and exact path readability (2). Suppose source evaluations return rational activations. For $\epsilon, \delta \in (0, 1)$, there is an algorithm using only sampled graphs, source activations and endpoint predictions that returns a ReLU vector-state student satisfying*

$$\operatorname{err}_{\mathcal{D}}(\widehat{T}, T) \leq \min\{1, 4\rho_{\text{end}} + \epsilon\}$$

with probability at least $1 - \delta$. The graph sample count is

$$m = O\left(\frac{v \log(2/\epsilon) + \log(6/\delta)}{\epsilon^2}\right),$$

where v is polynomial in $n, L, r, a, p, \sum_t (D_t + 1)$. If $\rho_{\text{end}} = 0$, the graph count improves to $O((v \log(2/\epsilon) + \log(2/\delta))/\epsilon)$ with the same polynomial-time guarantee. The algorithm's runtime and the student's size are polynomial in these parameters, $\log S$, the sample size, the encoded input length, and $\log(1/\epsilon)$, apart from source evaluation and graph sampling costs.

When the reference algorithm agrees with the source, this gives any prescribed endpoint error ϵ; otherwise the bound retains the term $4\rho_{\text{end}}$.

Proof Idea. The proof tracks linear combinations of learned features that approximate the unknown path indicators. Multiplying by the next literal extends a path by one test. Readability ensures that the extended path also has a linear expression in the source features, so regression toward those features can retain it while controlling the width $D_t + 1$. Repeating this argument preserves enough information to express the reference algorithm's final state. This argument adapts the path perspective of Boix-Adserà (2024). Appendix B.2 gives the ideal argument using exact linear algebra, and Appendix B.3 analyzes the regularized fitting pipeline. The proof uses the reference states only to establish that suitable combinations exist; the learner does not need to observe them. Appendices B.4 and B.5 establish the finite-precision implementation.

Sample Requirement. The proof controls generalization by bounding the complexity of graph losses computed by the learned circuits, using Goldberg and Jerrum (1995); see Appendix B.3. When $\rho_{\text{end}} = 0$, the *realizable case* here, the learner can fit every training graph correctly. A consistency bound, which controls the error of predictors with no training errors, then gives the faster $1/\epsilon$ rate (Blumer et al., 1989). Appendix B.6 proves this specialization (Corollary 4).

4.2 Approximate Paths and Observable Error Amplification

Approximate readability leaves residual error in each path readout. Reducing regularization does not remove this error. Later graph rounds can amplify it when combining learned features or copying one vertex's error to several others. Our bound therefore adds the errors introduced in each round after accounting for how the remaining rounds can amplify them.

We use the same weights as the fits: average equally over vertices within each sampled graph, then equally over graphs. The weighted RMS is the square root of this average of squared values. Two computable factors describe amplification at round t.

First, we consider the reconstruction factor Λ_t which measures amplification when expressing node values through the retained features. More precisely, to reconstruct a vector of sampled node values, we combine the retained feature columns using their weighted inner products with that vector as coefficients. Thus we can define Λ_t as the largest ratio of the reconstructed vector's maximum absolute value to the original vector's maximum absolute value, over nonzero vectors and inner iterations $0 \le k < r$.

Second, we bound the communication factor χ_t as is the largest ratio of the weighted RMS after one self or port lookup to the weighted RMS before it, over nonzero vectors and permitted lookups. A lookup copies the designated sender's value to each receiver, using zero for an absent port. Thus we also have $1 \le \chi_t \le \sqrt{n}$. Both factors depend only on the fitting graphs and learned features. See Appendix B.3 for more details on these factors.

Finally, we can put together the amplification bound. Since each literal is an input bit x or its complement $1 - x$, its coefficients have total absolute value at most two. Combining this coefficient bound with the tree bounds S, r and the two amplification factors gives the accumulated amplification bound

$$\mathcal{A}_m = Sr \sum_{t=1}^{L} \prod_{j=t+1}^{L} (2Sr\Lambda_j\chi_j).$$

The summand for round t follows its error through all later rounds; an empty product is one. The factors S and r account for the tree's leaves and path lengths.

The theorem also allows for sampling and numerical error. The tolerance ν bounds the increase from the population readout error ξ to its empirical value. The tolerance η controls generalization to fresh graphs, and ζ accounts for finite-precision error. The positive ridge parameters σ, μ, and θ regularize source normalization, regression on products, and retained-feature normalization, respectively, in the inner loop of Figure 2. The quantity ω below combines readout error, its sampling allowance, and the bias introduced by regularization. Recall also that τ and B_Z are the readout and source representation norm bounds, respectively.

Theorem 2 (Distillation under approximate path readability)**.** *Assume the interface of Section 3, (3), and its norm bounds. For $\nu, \eta, \delta \in (0, 1)$, let $\omega = \xi + \nu + \mu + \frac{1}{2}(\sigma\tau + \theta)$. There is a polynomial v in the architecture parameters such that, for a universal constant c, the graph count*

$$m \ge \max\left\{ \frac{2(1+\tau B_Z)^2}{\nu^2} \log \frac{6L(2S-1)}{\delta}, \frac{c}{\eta^2}\left[v \log \frac{2}{\eta} + \log \frac{6}{\delta} \right] \right\}$$

ensures, with probability at least $1 - \delta$, endpoint error at most $\min\{1, 4[\rho_{\text{end}} + \sqrt{n}\mathcal{A}_m\omega + \eta + \zeta]\}$.

Here $\zeta \in (0, 1)$ is a prescribed numerical loss allowance. For rational samples and positive rational regularizers, the algorithm uses finite precision and runs in polynomial time in their encoding lengths, the architecture and sample sizes, and $\log(1/\zeta)$. The returned ReLU student's size is polynomial in the same quantities.

The first sample-size requirement controls path residuals, and the second controls generalization of the final loss. The architecture parameters are those listed in Theorem 1.

Proof Idea. At each round, the proof adds the new readout and regularization errors to the amplified error from previous rounds. Iterating this bound gives $\mathcal{A}_m\omega$. The resulting approximation to the reference algorithm provides a comparison for the final prediction fit. A circuit-complexity

bound (Goldberg and Jerrum, 1995) then transfers the comparison from the fitting graphs to fresh graphs. The proof uses independent graphs as its sampling units. Appendix B.3 gives the statistical argument, and Appendix B.4 completes the finite-precision and ReLU implementation.

For one round, $\mathcal{A}_m = Sr$, but across several rounds, the factors multiply, and even small readout errors can lead to a large bound. A useful guarantee therefore requires both accurate readouts and controlled amplification; approximate readability alone does not ensure the latter. Appendix B.4 discusses using the computed bound to screen fits with excessive amplification.

4.3 What Endpoint Access Alone Cannot Provide

The existence of a small local computation does not by itself make distillation efficient. We show this using the standard assumption that secure pseudorandom functions exist and the resulting cryptographic obstruction to efficient learning (Goldreich et al., 1986; Kearns and Valiant, 1994). We use this assumption as a black box. Intuitively, a secret key determines a small local computation whose final predictions appear random if one only observes the output queries alone. An efficient learner that uses only these queries to build a student that reliably predicts fresh outputs would distinguish a randomly keyed function from a random one, contradicting the assumed security.

Theorem 3 (Informal endpoint-access separation)**.** *Assuming secure pseudorandom functions, there is a family of graph tasks computed by small shared local decision trees for which the following separation holds. With only final predictions and uninformative activations, no single randomized polynomial-time learner can guarantee endpoint error at most* $1/4$ *with probability at least* $2/3$ *for every task in the family, even with adaptive chosen-graph queries. With exactly readable path activations, Theorem 1 gives polynomial-time distillation to any prescribed accuracy and confidence from independent graph samples. Both cases allow the same polynomial-size ReLU message-passing student class, which contains an exact implementation of the target.*

The comparison fixes the graph distribution, final predictions, and student class; it changes which activations the source exposes. The learner receives only predictions and designated activations, and the deployed student makes no source calls. This construction separates the two choices of source access; it does not establish that our distillation algorithm is necessary for the positive result. Appendix C.1 gives the formal statement and proof, including the precise access model. Appendix C.2 separately gives unconditional sample lower bounds that hold even with readable paths.

5 Experiments

We test the representation conditions of Section 3.2 using two constructed sources: one with exactly readable decision-tree paths, and one with strictly positive path residuals. These isolate the learning mechanism of Theorem 1 and the error accumulation in Theorem 2. The learner receives graphs, source activations, and endpoint predictions, but no tree, path labels, intermediate states, or readouts. It learns its features and local computations using the regularized construction in Figure 2; the student then runs without the source. Reported numerical graph error counts a discrepancy from the source exceeding 10^{-8} at any vertex after applying the margin output $\mathrm{clip}_{[0,1]}(3h-1)$, where h is the student's score before this output transformation. Both sources match the reference endpoints, so $\rho_{\mathrm{end}} = 0$. Appendix D.1 gives the learning protocol; Appendices D.2 and D.3 give the respective experimental details and complete results.

Exactly readable paths. On eight-vertex graphs with one incoming port, the local update is parity of a raw bit, the current state, and the incoming state ($S = 8, r = 3$). An invertible rational

mixing of its fourteen nonempty path indicators and four nuisance coordinates makes every path linearly readable, as required by Section 3.2. The learner receives no readouts. We vary the fitting budget and horizon, using three fresh seeds and 2,048 test graphs per seed. Regularization is fixed after a pilot using 336 fitting draws and 1,024 additional validation graphs; these are excluded from the displayed budget.

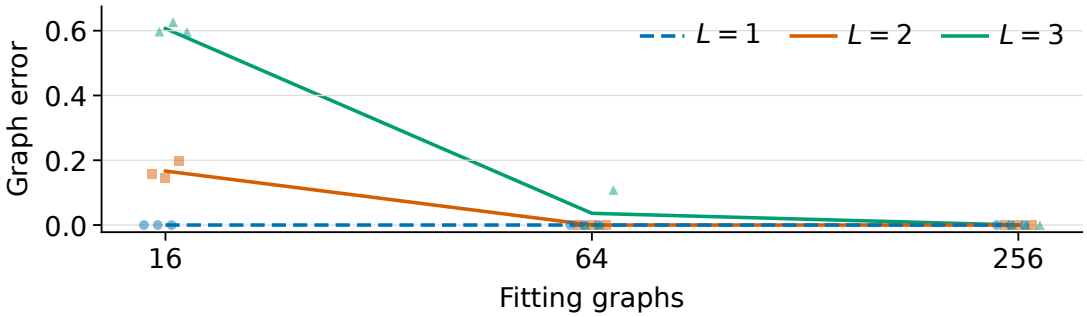


Figure 3: Exact-path learning curves. Lines show seed means; points show individual fits.

Figure 3 shows that increasing the sample budget resolves the harder, longer-horizon cases. This illustrates the mechanism of Theorem 1: readable paths support learning local computations without intermediate-state labels or a supplied local update. The rational source satisfies the exact premise, while fitting uses pilot-selected regularization and arithmetic products. These runs illustrate the construction rather than test its theoretical sample and precision schedules.

Approximate paths and amplification. We use the update $F(g, s, b) = gs + (1 - g)b$ $(S = 4, r = 2)$ on a directed three-cycle. Rationally mixed, perturbed activations have fixed, hidden nonempty-path readouts of norm $\sqrt{2}$ and pointwise residual ξ, satisfying (3) at every round. Exact rank checks exclude zero-residual recovery of these paths; the root remains exactly readable through the constant coordinate. The uniform distribution over all 64 Boolean input and initialization assignments permits exhaustive evaluation.

We use $\tau = 2$ and $\sigma = \mu = \theta = 2^{-20}$. Pointwise residuals give the empirical error term $\omega_m = \xi + 5 \cdot 2^{-21}$ directly, without a concentration allowance. Cycle routing has $\chi_t = 1$, so the learned reconstruction factors Λ_t control amplification across rounds. For three seeds at $m = 256$, Table 1 reports ReLU graph error and the expression inside the minimum in

$$B_{\text{measured}} = \min\{1, 4[\sqrt{3}\,\mathcal{A}_m\omega_m + \Delta_{\text{gap}} + \Delta_{\text{ReLU}}]\}.$$

Here Δ_{gap} is the positive population-minus-fitting loss gap, and Δ_{ReLU} is the maximum arithmetic-to-ReLU score discrepancy over this population (Appendix D.3).

L	ξ	Error in each seed	Uncapped bound range
2	2^{-20}	0/64	0.00778–0.00830
2	2^{-16}	0/64	0.04114–0.04385
2	2^{-12}	0/64	0.57487–0.61265
3	2^{-20}	0/64	0.38650–0.47591
3	2^{-16}	0/64	1.94515–2.49491
3	2^{-12}	0/64	27.13496–34.88952

Table 1: Approximate-path distillation.

Table 1 shows accurate endpoints with strictly positive representation error. The calculated bound becomes looser as the residual or horizon increases, reflecting the $\sqrt{n}\mathcal{A}_m\omega$ term in Theorem 2; in these runs, the longer horizon makes the bound more sensitive to readout error. Some bounds become vacuous despite accurate predictions, and smaller fitting budgets can fail (Appendix D.3).

6 Conclusion and Discussion

We show that source representations can support learning a local graph computation from final predictions without intermediate-state labels. We prove this when a reference algorithm matches the source endpoints, its decision-tree paths are linearly readable, and the student can carry learned vectors through a known communication interface. Our algorithm learns the features and local circuits jointly. For exact readability of rational source activations, it constructs a polynomial-size student in polynomial time using finite precision, without a supplied readout-norm bound.

The computational lower bound identifies a limit of endpoint access for the same student class. Under pseudorandom-function security, no single efficient learner can distill every task in the constructed family from endpoints and designated uninformative features, even though readable path features enable efficient distillation by our upper bound. This separation permits arbitrary efficient students and adaptive graph queries. It depends on the specified source interface: access to source parameters or other internal activations can reveal information that the lower bound withholds.

Under approximate readability, our bound contains a residual that decreasing regularization does not remove. We track its propagation through a computable error-amplification bound. Controlling this quantity across rounds is necessary for this error bound to be informative.

The experiments in Section 5 illustrate learning with exactly readable paths and the approximate bound's amplification cost. The remaining questions are which natural graph tasks admit useful path representations, which conditions control amplification across rounds, and how implementations can reduce the sample and computational costs of the construction.

Use of generative AI

We developed the original version of this work without generative AI. In later revisions, we asked Codex and Claude to address gaps that remained open. We formulated all research questions and chose the results to present, supplied candidate theorem statements, attempted proofs, and proof sketches, and directed the tools to review the relevant literature. We studied their outputs through many iterations and rewrote the paper, arriving at the theorem statements and proofs presented here. In particular, Codex GPT-6 Astra suggested the hardness lower bound with minimal input from us.

In these revisions, we wrote the manuscript with help from Codex and Claude to improve its exposition. We designed the experiments; the AI tools wrote the code and ran them. The two schematic figures were also created with AI tools under our direction. We checked all theoretical results, experiments, and writing, and take full responsibility for the paper.

References

Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes. In *International Conference on Learning Representations, Workshop Track*, 2017. URL https://openreview.net/forum?id=HJ4-rAVtl.

Francis R. Bach and Michael I. Jordan. Kernel independent component analysis. *Journal of Machine Learning Research*, 3:1–48, 2002. URL https://www.jmlr.org/papers/volume3/bach02a/bach02a.pdf.

Gregor Bachmann, Sotiris Anagnostidis, and Thomas Hofmann. Scaling MLPs: A tale of inductive bias. In *Advances in Neural Information Processing Systems*, volume 36, pages 60821–60840. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/bf2a5ce85aea9ff40d9bf8b2c2561cae-Abstract-Conference.html.

Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens. arXiv preprint arXiv:2303.08112v6, 2023. URL https://arxiv.org/abs/2303.08112v6. Version 6, revised November 2025; first posted in 2023.

Beatrice Bevilacqua, Kyriacos Nikiforou, Borja Ibarz, Ioana Bica, Michela Paganini, Charles Blundell, Jovana Mitrovic, and Petar Veličković. Neural algorithmic reasoning with causal regularisation. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 2272–2288. PMLR, 2023. URL https://proceedings.mlr.press/v202/bevilacqua23a.html.

Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K. Warmuth. Learnability and the Vapnik–Chervonenkis dimension. *Journal of the ACM*, 36(4):929–965, 1989. doi: 10.1145/76359.76371. URL https://mwarmuth.bitbucket.io/pubs/J14.pdf.

Enric Boix-Adserà. Towards a theory of model distillation. arXiv preprint arXiv:2403.09053, 2024. URL https://arxiv.org/abs/2403.09053.

Johann Brehmer, Sönke Behrends, Pim de Haan, and Taco Cohen. Does equivariance matter at scale? *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=wilNute8Tn.

Richard P. Brent and Paul Zimmermann. *Modern Computer Arithmetic*, volume 18 of *Cambridge Monographs on Applied and Computational Mathematics*. Cambridge University Press, 2010. doi: 10.1017/CBO9780511921698. URL https://members.loria.fr/PZimmermann/mca/pub226.html.

Andrzej Ehrenfeucht, David Haussler, Michael Kearns, and Leslie Valiant. A general lower bound on the number of examples needed for learning. *Information and Computation*, 82(3):247–261, 1989. doi: 10.1016/0890-5401(89)90002-3. URL https://www.cis.upenn.edu/~mkearns/papers/lower.pdf.

Nikhil Garg, Jon Kleinberg, and Kenny Peng. How many features can a language model store under the linear representation hypothesis? In *Proceedings of Thirty Ninth Conference on Learning Theory*, volume 336 of *Proceedings of Machine Learning Research*, pages 5358–5376. PMLR, 2026. URL https://proceedings.mlr.press/v336/garg26a.html.

Atticus Geiger, Hanson Lu, Thomas Icard, and Christopher Potts. Causal abstractions of neural networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 9574–9586. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/4f5c422f4d49a5a807eda27434231040-Abstract.html.

Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272. PMLR, 2017. URL https://proceedings.mlr.press/v70/gilmer17a.html.

Paul W. Goldberg and Mark R. Jerrum. Bounding the Vapnik–Chervonenkis dimension of concept classes parameterized by real numbers. *Machine Learning*, 18:131–148, 1995. doi: 10.1007/BF00993408. URL https://www.cs.ox.ac.uk/people/paul.goldberg/papers/GJ-ML95.pdf.

Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, 1986. doi: 10.1145/6490.6503. URL https://www.wisdom.weizmann.ac.il/~oded/X/ggm-jacm.pdf.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL https://www.deeplearningbook.org/.

Nicholas J. Higham. Computing the polar decomposition—with applications. *SIAM Journal on Scientific and Statistical Computing*, 7(4):1160–1174, 1986. doi: 10.1137/0907079. URL https://eprints.maths.manchester.ac.uk/694/1/high86p.pdf.

Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, PA, second edition, 2002. doi: 10.1137/1.9780898718027. URL https://epubs.siam.org/doi/book/10.1137/1.9780898718027.

Nicholas J. Higham. *Functions of Matrices: Theory and Computation*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2008. doi: 10.1137/1.9780898717778. URL https://epubs.siam.org/doi/book/10.1137/1.9780898717778.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015. URL https://arxiv.org/abs/1503.02531.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. TinyBERT: Distilling BERT for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4163–4174. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.findings-emnlp.372. URL https://aclanthology.org/2020.findings-emnlp.372/.

Erich L. Kaltofen, Michael Nehring, and B. David Saunders. Quadratic-time certificates in linear algebra. In *Proceedings of the 36th International Symposium on Symbolic and Algebraic Computation*, pages 171–176. ACM, 2011. doi: 10.1145/1993886.1993915. URL https://kaltofen.math.ncsu.edu/bibliography/11/KNS11.pdf.

Michael Kearns and Leslie Valiant. Cryptographic limitations on learning Boolean formulae and finite automata. *Journal of the ACM*, 41(1):67–95, 1994. doi: 10.1145/174644.174647. URL https://www.cis.upenn.edu/~mkearns/papers/crypto.pdf.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017. URL https://openreview.net/forum?id=SJU4ayYgl.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25.

Curran Associates, Inc., 2012. URL https://proceedings.neurips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.

Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539. URL https://www.nature.com/articles/nature14539.

Roi Livni, David Lehavi, Sagi Schein, Hila Nachlieli, Shai Shalev-Shwartz, and Amir Globerson. Vanishing component analysis. In *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pages 597–605. PMLR, 2013. URL https://proceedings.mlr.press/v28/livni13.html. Cycle 1.

Roi Livni, Shai Shalev-Shwartz, and Ohad Shamir. An algorithm for training polynomial networks. arXiv preprint arXiv:1304.7045v2, 2014. URL https://arxiv.org/abs/1304.7045v2. Version 2; originally posted in 2013.

Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 39643–39666. PMLR, 2024. URL https://proceedings.mlr.press/v235/park24c.html.

Nico Pelleriti, Max Zimmer, Elias Samuel Wirth, and Sebastian Pokutta. Approximating latent manifolds in neural networks via vanishing ideals. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48734–48761. PMLR, 2025. URL https://proceedings.mlr.press/v267/pelleriti25a.html.

Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. FitNets: Hints for thin deep nets. In *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1412.6550.

Ian Tenney, Dipanjan Das, and Ellie Pavlick. BERT rediscovers the classical NLP pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601. Association for Computational Linguistics, 2019. doi: 10.18653/v1/P19-1452. URL https://aclanthology.org/P19-1452/.

Petar Veličković, Adrià Puigdomènech Badia, David Budden, Razvan Pascanu, Andrea Banino, Misha Dashevskiy, Raia Hadsell, and Charles Blundell. The CLRS algorithmic reasoning benchmark. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 22084–22102. PMLR, 2022. URL https://proceedings.mlr.press/v162/velickovic22a.html.

Elias S. Wirth and Sebastian Pokutta. Conditional gradients for the approximately vanishing ideal. In *Proceedings of the 25th International Conference on Artificial Intelligence and Statistics*, volume 151 of *Proceedings of Machine Learning Research*, pages 2191–2209. PMLR, 2022. URL https://proceedings.mlr.press/v151/wirth22a.html.

Keyulu Xu, Jingling Li, Mozhi Zhang, Simon S. Du, Ken-ichi Kawarabayashi, and Stefanie Jegelka. What can neural networks reason about? In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=rJxbJeHFPS.

Dmitry Yarotsky. Error bounds for approximations with deep ReLU networks. *Neural Networks*, 94: 103–114, 2017. doi: 10.1016/j.neunet.2017.07.002. URL https://arxiv.org/abs/1610.01145.

Appendix contents

Appendix A extends the related-work discussion. Appendices B and C prove the main upper and lower bounds. Appendix D gives experimental details.

A Extended Related Work **17**
A.1 Representation-Guided Distillation and Feature Construction 17
A.2 Local Learning and Intermediate Supervision 17

B Proofs for Distillation from Source Activations and Endpoints **18**
B.1 Interface and Empirical Representation 18
B.2 Exact Span Propagation 19
B.3 Proof of Theorem 2: Statistical Guarantee 20
B.4 Numerical Computation and ReLU Compilation 25
B.5 Exact Readability Yields a Polynomial-Time Learner 27
B.6 A Fast Graph-Sample Rate for Exact Endpoint Agreement 29

C Lower Bounds under Specified Source Access **30**
C.1 A Computational Separation for the Same Student Class 30
C.2 Unconditional Requirements on the Number of Sampled Graphs 34

D Experimental Details **36**
D.1 Learning Protocol and Numerical Evaluation 36
D.2 Regularized Learning with Exactly Readable Parity Paths 37
D.3 Exhaustive Finite-Population Check of the Approximate Bound 38

A Extended Related Work

The comparisons in Section 1.2 concern two parts of our learning problem: how source activations guide feature construction, and which supervision is available for the local computation. We expand those comparisons here to distinguish our representation assumption from supplied intermediate labels and from identification of the source's own computation.

A.1 Representation-Guided Distillation and Feature Construction

Our path-readability assumption follows the decision-tree representation condition of Boix-Adserà (2024, Section 3.2). That condition makes the conjunctions along root-to-node paths linearly readable from a neural representation. The resulting learner first finds readable conjunctions and then assembles an explicit decision tree (Boix-Adserà, 2024, Theorem 3.6 and its proof). For arbitrary input distributions, its runtime has exponential dependence on tree depth; the uniform Boolean-cube case admits polynomial dependence on tree size. The general distillation framework supplies a representation of the source function, such as its network weights (Boix-Adserà, 2024, Definition 2.2).

Our learner retains the readable-path premise but receives only the specified source activations and final predictions on sampled graphs. It returns a vector-state student whose local update modules may differ across rounds. This larger student class lets the learner preserve linear combinations of useful paths without choosing individual paths or assembling a decision tree. Theorem 1 therefore gives a polynomial-time result for this circuit class under exact readability of rational activations; it does not resolve proper decision-tree learning with polynomial dependence on depth. The graph setting adds a further requirement: the retained features must continue to express the unknown reference states after communication and local updates.

To meet this requirement, we adapt an established way of constructing polynomial features. The Basis Learner of Livni et al. (2014) successively forms products, selects a basis of feature evaluations, and fits its output by convex optimization. Our learner uses products to extend local paths and regularized regression toward source activations to retain at most $D_t + 1$ features at round t. Readability is what allows this restricted feature space to preserve the reference computation across graph rounds. Thus the product construction is inherited from polynomial-feature learning, whereas its role here is to maintain a local computation without observing that computation's intermediate states.

A.2 Local Learning and Intermediate Supervision

The absence of intermediate-state labels distinguishes our learning problem from supervised learning of aligned modules. Xu et al. (2020, Theorem 3.6) prove a composition guarantee when modules are learned sequentially with auxiliary labels, the learning algorithm is stable under input perturbations, and the learned modules satisfy Lipschitz bounds. Those labels specify the intermediate functions to be learned. Our learner instead constructs its own vector coordinates from source activations. The readability assumption supplies information that supports this construction, while the approximate theorem tracks the resulting error amplification explicitly.

Intermediate supervision itself can take different forms. FitNets uses a teacher's hidden representations as targets for a student's intermediate layer (Romero et al., 2015). CLRS provides algorithmic traces as intermediate hints (Veličković et al., 2022), and Bevilacqua et al. (2023) use supplied trajectories to construct augmentations that preserve an algorithm's next step. These procedures clarify the information distinction in our setting: the learner observes teacher activations,

but neither labeled reference states nor the readouts that would recover path indicators from those activations. Its guarantee concerns final predictions and permits learned states that differ from the reference states.

This freedom in the student's states also limits the interpretation of the result. Successful decoding alone need not identify information that the source uses causally; Geiger et al. (2021) demonstrate this distinction and study causal alignment through interventions. Our readability assumption asserts the existence of useful linear readouts, and our theorem constructs a student that can exploit the information they describe. It does not identify the source's causal algorithm. With the information and output requirements fixed, Appendices B.2 and B.3 show how the learned feature spaces retain enough of the reference computation to predict its endpoints.

B Proofs for Distillation from Source Activations and Endpoints

Theorems 1 and 2 establish endpoint guarantees without intermediate-state labels. The proof maintains a space that contains, or approximates, the unknown reference states. We first establish this invariant using exact span intersections. We then replace intersections by regularized regression, bound the resulting error, and control numerical precision and ReLU compilation. Exact readability and rational source values then yield the polynomial bit-time guarantee of Theorem 1, without an assumed bound on error amplification. The final subsection improves the sample rate when the reference algorithm agrees exactly with the source endpoints. These arguments adapt the path-representation approach of Boix-Adserà (2024). None of our results assumes that the learner receives the paths, their readouts, or their values on the training graphs.

B.1 Interface and Empirical Representation

We recall the interface to specify what the student can compute during the proofs. Each graph has between one and n vertices, a Boolean local attributes, and at most p labeled incoming ports per vertex. A port identifies one sender, and a missing sender contributes zero. The initialization $z^0 = \mathrm{Init}(G)$ is known and Boolean. An unknown shared Boolean decision tree generates the reference algorithm's states $z^1, \dots, z^L$, as in Section 3.1. Its local input vector directly concatenates the a raw attributes at the vertex, its own state, and its p incoming-port states, with zero padding for missing ports. Thus it has $q = a + p + 1$ coordinates, as in Section 3.1. Port labels are part of the input graph and move with their vertices under relabeling.

For each round t, ordinary source evaluation supplies a fixed node-indexed vector $Z_u^t(G) \in \mathbb{R}^{D_t}$. All source representations and the reference algorithm are fixed before sampling. A path P in its local tree defines a Boolean indicator a_P on local input vectors. Exact path readability means that $a_P(C_G(z^{t-1})_u)$ is an unknown linear combination of $(1, Z_u^t(G))$, almost surely. The activation selection is fixed independently of the sample; its index t specifies a target graph round, not necessarily a source layer. The algorithm inspects activations and endpoints on sampled graphs only.

Fix independent fitting graphs $G_1, \dots, G_m$ and stack all their vertices into rows. A state or feature is then a real vector indexed by pairs (i, u) with $u \in V(G_i)$. All empirical vectors and feature matrices use this row index. The student shares each learned local expression across vertices, while its vector states and local modules may differ across rounds. These are the vector states and local update modules defined in Section 3.

B.2 Exact Span Propagation

The proof idea in Section 4 is that learned features need only express the unknown reference states through linear combinations. Exact span intersections isolate this idea before we analyze regularization. We show that multiplying available features and retaining their intersection with the source span preserves every reference state on the fitting graphs. This construction explains the invariant; the subsequent subsections establish the statistical and finite-precision guarantees of the main theorems.

At round t, let V_t be the column span of the stacked source matrix $(1, Z^t)$. Let U^t be the matrix of available student inputs: each row contains a constant, raw attributes, and the previous learned features at the vertex and its incoming ports. At the first round, those previous features are $(1, z^0)$. Writing $D_0 = 1$ for this initialization, pad each row with zeros to the common width

$$p_t := 1 + a + (p+1)(D_{t-1} + 1).$$

We seek a subspace W_t that contains the reference state while remaining inside the source span, whose dimension is at most $D_t + 1$.

To construct this subspace, start with $W_t = \mathrm{span}\{\mathbf{1}\}$. Given a basis of W_t, compute

$$V_t \cap \mathrm{span}\{U_j^t \odot w : 1 \le j \le p_t,\ w \in W_t\}, \tag{4}$$

where U_j^t is the jth column and $\odot$ denotes entrywise multiplication. If the intersection contains a vector outside W_t, add one to the basis and repeat. The constant column of U^t ensures that the product span contains W_t. Each addition increases dimension inside V_t, so at most D_t additions are possible. For each added basis vector, store its linear expression in the products of available inputs and earlier basis vectors. These expressions define the learned features on new graphs without source access; zero padding gives a vector of width $D_t + 1$.

We prove the desired invariant by induction on graph rounds and then on tree paths. The initial reference state is available exactly. Suppose that z^{t-1} belongs to the preceding learned span. Its coefficients apply at every vertex, so each self or port lookup preserves that linear expression, including zero padding at missing ports. The raw attributes are already columns of U^t, and the self and port reference states lie in its span by the induction hypothesis. Direct concatenation therefore places every reference input bit in that span.

With these input bits available in the span, we can extend each path by one test. Suppose a root-to-node path P ends at an internal node that tests an input bit, and let b be the vector of that bit's values on the fitting graphs. The positive child is reached when the tested bit equals one, and the negative child when it equals zero. Write P^+ and P^- for the paths obtained by extending P to these two children. Their indicator vectors satisfy

$$a_{P^+} = a_P \odot b, \qquad a_{P^-} = a_P \odot (\mathbf{1} - b) = a_P - a_{P^+}.$$

The empty-path indicator is the constant vector in W_t. If a_P belongs to the final W_t, then $b \in \mathrm{span}(U^t)$ places a_{P^+} in the product span in (4). Exact path readability also places a_{P^+} in V_t. At termination, their intersection has no direction outside W_t, so $a_{P^+} \in W_t$. The displayed identity then gives $a_{P^-} \in W_t$. Induction through the tree places every leaf indicator in W_t. Since each input reaches exactly one leaf, summing the indicators of leaves labeled one gives z^t, proving the round invariant.

At the last round, a linear combination of the learned features thus reproduces z^L on the fitting graphs. When $\rho_{\mathrm{end}} = 0$, this also reproduces the source endpoints. The span intersections require polynomially many exact linear-algebra operations, but this observation does not bound their numerical sensitivity or bit cost. Regularized regression addresses those issues while replacing exact containment by a quantitative approximation bound.

B.3 Proof of Theorem 2: Statistical Guarantee

The exact-span argument identifies the paths that the feature space must preserve, but span intersections can change discontinuously under small perturbations. To handle approximate readability, we replace intersections by regularized regression. We first describe the regression and the stored local updates, then bound the error in one path extension and propagate it across rounds. Finally, we compare the endpoint fit with the approximate reference state and control its loss on new graphs. This proves the statistical part of Theorem 2; Appendix B.4 supplies the numerical and ReLU implementation.

Throughout this proof, retain the assumptions of Section 3. Let the reference algorithm use a tree with at most $S \geq 1$ leaves and depth at most $r \geq 1$. Recall that ρ_{end} is its whole-graph endpoint disagreement with the source. Direct concatenation makes each literal either a raw attribute, a self or port state, or the complement of one of these bits. Its affine coefficients, including the constant term, therefore have sum of absolute values at most two. We use this fixed bound throughout the proof; no additional input transformation is supplied to the learner.

For the representation assumption, let $\tau \geq 0$ bound the readout norm, $\xi \geq 0$ bound its population root mean squared residual, and $B_Z \geq 1$ bound the augmented source-feature norm. For every path P in the reference algorithm's tree and round t, assume that a fixed unknown coefficient vector w_P^t satisfies

$$\|w_P^t\|_2 \leq \tau, \qquad \mathbb{E}_{G\sim\mathcal{D}} \frac{1}{|V(G)|} \sum_u \left| a_P(C_G(z^{t-1})_u) - (w_P^t)^\top (1, Z_u^t(G)) \right|^2 \leq \xi^2.$$

Assume also $\|(1, Z_u^t(G))\|_2 \leq B_Z$ almost surely. These are the fixed readouts from the approximate-readability assumption (3); the learner receives the norm bounds τ, B_Z, but not the readouts. The reference states in this assumption still follow the same decision-tree update exactly. Approximation enters through their path readouts and through the reference algorithm's endpoint disagreement with the source.

Weighted Ridge Regression Preserves Coefficient Control. The regression norm must respect the graph-level sampling unit. We therefore give each graph equal weight and its vertices equal conditional weight. For node-value columns f, g, write $f_{iu} = f(G_i, u)$ and define the empirical inner product and norm by

$$\langle f, g\rangle_m = \frac{1}{m} \sum_i \frac{1}{|V(G_i)|} \sum_u f_{iu} g_{iu}, \qquad \|f\|_m^2 = \langle f, f\rangle_m.$$

Let Ω be the diagonal matrix with entry $1/(m|V(G_i)|)$ for row (i, u). A feature matrix X has weighted adjoint $X^* = X^\top \Omega$, so that $X^* f$ lists the weighted inner products of f with its columns. To preserve the empty-path indicator exactly, separate the constant direction before regularizing. Let P_1 be the orthogonal projection onto that direction, given by $P_1 f = \mathbf{1}\langle \mathbf{1}, f\rangle_m$, and let $X_\perp = (I - P_1)X$ be the centered feature matrix. For a ridge parameter $\lambda > 0$, define the normalized feature matrix $Q_\lambda(X)$ and its ridge reconstruction operator $R_\lambda(X)$ by

$$\begin{aligned} Q_\lambda(X) &= [\mathbf{1}, X_\perp (X_\perp^* X_\perp + \lambda^2 I)^{-1/2}], \\ R_\lambda(X) &= Q_\lambda(X) Q_\lambda(X)^* = P_1 + X_\perp (X_\perp^* X_\perp + \lambda^2 I)^{-1} X_\perp^*. \end{aligned}$$

Here λ^2 is the ridge penalty coefficient: $R_\lambda(X)$ returns the fitted values when applied to a response vector, while preserving the constant direction without penalization. All singular values in this calculation use the empirical row norm: they are the ordinary singular values after left multiplication by $\Omega^{1/2}$. The nonconstant singular values of $Q_\lambda(X)$ have the form $s/\sqrt{s^2 + \lambda^2} \leq 1$, where s

ranges over the singular values of $\Omega^{1/2}X_\perp$. Its constant column has empirical norm one and is orthogonal to the others. Thus $Q_\lambda(X)$ is a contraction from the Euclidean coefficient space to the empirical norm, and both $R_\lambda(X)$ and $I - R_\lambda(X)$ are contractions in the empirical norm. In particular, $\|Q_\lambda(X)^* a\|_2 \le 1$ for a Boolean vector a. Regularized covariance contractions have this feature-comparison role in Bach and Jordan (2002, Section 4.3). The residual estimate needed here follows directly from the same singular-value calculation: for any coefficient vector c,

$$\|(I - R_\lambda(X))Xc\|_m \le \frac{\lambda}{2}\|c\|_2, \tag{5}$$

because each singular value $s \ge 0$ satisfies $\lambda^2 s/(s^2+\lambda^2) \le \lambda/2$. An additional constant term contributes no residual.

Inner Iterations and Stored Updates. We use these operators to alternate product expansion with regression toward the source representation. Choose positive ridge parameters σ, μ, θ. At round t, the row of U^t for sampled vertex (G_i, u) collects a constant, raw attributes, and the preceding learned feature vectors at u and its incoming ports. The known initialization supplies $(1, z^0)$ at the first round. Keep U^t fixed throughout the inner iterations; these computations require no additional communication.

Before the inner iterations, compute $Q_t^Z = Q_\sigma(Z^t)$ once for this round, and denote its nonconstant columns by $\overline{Q}_t^Z$. This operation subtracts the weighted source-column means and applies regularized normalization with parameter σ. The resulting columns serve as targets throughout the round. Let $Q_{t,k}$ record the retained features after inner iteration k, with the same sampled rows as U^t, and initialize $Q_{t,0} = \mathbf{1}$. For $k = 1, \dots, r$, let $A_{t,k}$ be the product matrix and $M_{t,k}$ the matrix of fitted source targets, and compute

$$\begin{aligned} A_{t,k} &= [U_j^t \odot (Q_{t,k-1})_h]_{j,h}, \\ M_{t,k} &= R_\mu(A_{t,k})\overline{Q}_t^Z, \\ Q_{t,k} &= Q_\theta(M_{t,k}). \end{aligned}$$

The first equation forms every pointwise product of an available input column and a previously retained feature. The second fits the normalized source targets from these products by ridge regression with parameter μ. The third centers and normalizes the fitted columns with parameter θ, while retaining the constant exactly to preserve the empty path. The resulting $Q_{t,k}$ has $D_t + 1$ columns, allowing zero columns: one for each source coordinate and one constant.

To deploy these columns on new graphs, observe that every nonconstant column is an affine combination of products in $A_{t,k}$, so the student can evaluate it without querying the source. Store the expressions from all inner iterations and clip their features to $[-B_c, B_c]$, where $B_c := mn$. Each ideal feature column has empirical norm at most one and each row weight is at least $1/(mn)$, so every fitting entry has magnitude at most $\sqrt{mn} \le B_c$. Thus clipping changes no fitting value. It bounds every feature on new graphs as well. Pass only $Q_{t,r}$ to the next message-passing round. Its rows record the vertex feature vectors before ReLU approximation. The implementation in Appendix B.4 approximates these expressions to produce the student states $\mathbf{h}_u^t$ shown in Figure 1(a).

After constructing the features, we use the endpoint labels to fit the final predictor. Set the coefficient bound $R_0 := \lceil \sqrt{D_L + 1} \rceil$, which will make the approximate reference state feasible. Let $\gamma \in \mathbb{R}^{D_L+1}$ be the final readout vector, and let b_i bound the maximum absolute fitting loss on graph G_i. Fit them by the linear program

$$\min_{\gamma, b_1, \dots, b_m} \frac{1}{m}\sum_i b_i, \quad |\gamma^\top Q_{L,r}(i,u) - T_u(G_i)| \le b_i, \quad b_i \ge 0, \quad \|\gamma\|_1 \le R_0. \tag{6}$$

There is one displayed loss constraint for every sampled vertex. The scores need not lie in $[0,1]$, since the approximation to the Boolean reference state may leave that interval. Clipping will instead enter the bounded statistical loss.

Computing the Amplification Factors. To analyze how the constructed features propagate errors, we express the two amplification factors from Section 4 as matrix norms. For each retained-feature matrix, define $Q^*_{t,k} = Q_{t,k}^\top \Omega$ and the reconstruction operator $R_{t,k} := Q_{t,k} Q^*_{t,k}$. Acting on a node-value column, the adjoint computes its weighted inner products with the feature columns, and $Q_{t,k}$ recombines those columns with the resulting coefficients. Regularization shrinks some components, so $R_{t,k}$ need not be an orthogonal projection. The reconstruction factor is

$$\Lambda_t := \max_{0 \le k < r} \|R_{t,k}\|_{\infty \to \infty}.$$

This induced matrix norm is the maximum absolute row sum, equivalently the largest possible ratio of output to input maximum absolute value.

The communication factor must also account for copying an error through the graph. Let $\mathcal{P}_t$ be the set of self and incoming-port routing matrices on sampled vertices. Each matrix copies the designated sender's value to its receiver, with zero for an absent port. Define

$$\chi_t := \max_{P \in \mathcal{P}_t} \sup_{f \neq 0} \frac{\|Pf\|_m}{\|f\|_m}.$$

Self routing gives $\chi_t \ge 1$. Within a graph, all vertices have the same weight and each sender can be copied at most n times, so $\chi_t \le \sqrt{n}$. Both factors are computable from the sampled graphs and retained features, without reference-state labels.

One Path Extension. To bound the representation error, we first extend one approximate path indicator by a single literal. Write $E_t := \|z^t - R_{t,r} z^t\|_m$ for the empirical reference-state approximation error at round t, and set $E_0 = 0$ because the initialization is known.

At inner iteration k, consider a path indicator a with approximation $q = R_{t,k-1} a$ and error $\|a - q\|_m \le e$. Its coefficient vector $Q^*_{t,k-1} a$ has norm at most one, and $\|q\|_\infty \le \Lambda_t$. Replace each previous reference state in the next literal ℓ by its regularized approximation. The resulting vector v lies in the column span of U^t. A reconstructed state has coefficient norm at most one; taking its complement adds the constant column. Thus v has coefficient norm at most two, and conservatively

$$\|\ell - v\|_m \le 2\chi_t E_{t-1}.$$

At the first round the states are available exactly. Tensoring the coefficient vectors shows that qv lies in the column span of $A_{t,k}$ with coefficient norm at most 2. Since ℓ is Boolean,

$$\|a\ell - qv\|_m \le e + 2\Lambda_t \chi_t E_{t-1}.$$

The contraction property and (5) bound the product-regression residual by this expression plus μ.

The source fit must retain this extended path. Write $a' = a\ell$ for its indicator, let ξ_m bound the empirical RMS residuals of all fixed path readouts, and let $R_Z := Q^Z_t (Q^Z_t)^*$ be the normalized-source reconstruction operator. The readout assumption and (5) give

$$\|(I - R_Z) a'\|_m \le \xi_m + \sigma\tau/2.$$

The source coefficient vector $(Q^Z_t)^* a'$ has norm at most one. Both regressions preserve constants, so $R_\mu(A_{t,k}) R_Z a'$ is a linear combination of $[\mathbf{1}, M_{t,k}]$ with coefficient norm at most one. Apply (5) once more, then use contraction of the product regression, to obtain

$$\begin{aligned} \|(I - R_{t,k}) a'\|_m &\le \|a' - R_\mu(A_{t,k}) R_Z a'\|_m + \theta/2 \\ &\le e + 2\Lambda_t \chi_t E_{t-1} + \xi_m + \mu + \tfrac{1}{2}(\sigma\tau + \theta). \end{aligned}$$

Shorter paths satisfy the same estimate by using the literal one; the product matrix includes the previous representation. Starting from the exact empty path, r inner iterations approximate every leaf. Since z^t is the sum of at most S positive-leaf indicators, linearity and the triangle inequality give

$$E_t \le Sr\big[2\Lambda_t\chi_t E_{t-1} + \xi_m + \mu + \tfrac{1}{2}(\sigma\tau+\theta)\big]. \tag{7}$$

To record the amplification across rounds, define

$$\mathcal{A}_m := Sr\sum_{t=1}^{L}\prod_{j=t+1}^{L}(2Sr\Lambda_j\chi_j). \tag{8}$$

An empty product is one. If every empirical path residual is at most $\xi+\nu$, define the combined readout and regularization allowance by $\omega := \xi+\nu+\mu+\frac{1}{2}(\sigma\tau+\theta)$. Iterating (7) gives $E_L \le \mathcal{A}_m\omega$. We can compute this amplification bound from observed matrices and known interface bounds, without access to the unknown states or readouts.

From Population Readability to the Fitting Sample. The preceding recurrence requires a bound on fitting residuals, whereas the assumption bounds population residuals. Set $M_0 := 1+\tau B_Z$, an almost-sure bound on a path readout residual. For a fixed path and round, the graph-average squared residual lies in $[0, M_0^2]$, has expectation at most ξ^2, and has variance at most $M_0^2\xi^2$. Bernstein's inequality bounds its empirical mean by $\xi^2 + M_0\xi\sqrt{2s/m} + 2M_0^2 s/(3m)$ with failure probability at most e^{-s}. This is no larger than $(\xi + M_0\sqrt{2s/m})^2$. Taking $s = \log(6L(2S-1)/\delta)$ and a union bound over at most $L(2S-1)$ path–round pairs gives

$$\xi_m \le \xi + M_0\sqrt{\frac{2\log(6L(2S-1)/\delta)}{m}}$$

simultaneously, except with probability $\delta/3$. Consequently,

$$m \ge \frac{2(1+\tau B_Z)^2}{\nu^2}\log\frac{6L(2S-1)}{\delta}$$

ensures the premise used in the recurrence. The union bound ranges over the paths of the fixed reference algorithm's tree; the learner does not enumerate them. Independence is required only across graphs.

Endpoint Fitting and Generalization. We first compare the endpoint LP with the approximation to the reference state, then transfer its empirical loss to new graphs. The final approximation $h_\star = R_{L,r}z^L$ has coefficient vector $\gamma_\star = Q^*_{L,r}z^L$ of norm at most one. Therefore $\|\gamma_\star\|_1 \le \sqrt{D_L+1} \le R_0$, making it feasible in (6). Let $\widehat{\rho}_{\mathrm{end}}$ be the fitting fraction of graphs on which z^L and T differ. The maximum absolute error of $h_\star$ on each graph, averaged over fitting graphs, is at most

$$\widehat{\rho}_{\mathrm{end}} + \frac{1}{m}\sum_i \max_u |(h_\star)_{iu} - z^L_{iu}| \le \widehat{\rho}_{\mathrm{end}} + \sqrt{n}E_L.$$

The last inequality follows by Cauchy–Schwarz within and between graphs. The LP optimum is no larger.

Clipping cannot increase absolute error relative to a Boolean label, so the fitted scores also control a bounded loss. For any score h in the stored architecture, define that graph loss by

$$\ell_h(G) := \max_{u\in V(G)} |\operatorname{clip}_{[0,1]}(h_u(G)) - T_u(G)|.$$

Its empirical value is bounded by the raw fitting loss. Let K count all coefficients in the fixed architecture of affine, product, and clipping operations, including intercepts and the final score. For this capacity calculation, encode each graph using its raw attributes, known initialization, port selectors, source endpoint labels, and vertex-presence masks, padding to n vertices. This encoding is fixed independently of fitting. Let J bound the arithmetic operations and comparisons needed to test whether the loss exceeds a given threshold on such an encoded graph. Both counts are polynomial in $n, L, r, a, p, \sum_t (D_t + 1)$ and independent of m. The clipping level mn is fixed once the sample size is chosen and does not increase either count.

A loss level test uses K fitted coefficients and one real threshold. Goldberg and Jerrum (1995, Theorem 2.3) bound its VC dimension by

$$v := 1 + \lceil 2(K+1)(2J + \log_2(8e)) \rceil .$$

This counts evaluation of the fitted circuit, rather than the matrix operations that selected its coefficients. In particular, the source labels are part of the graph encoding, so the bound does not charge evaluation of the source itself. Only the loss evaluation uses these label coordinates; the student score still depends only on graph inputs and its fitted coefficients.

To pass from this capacity bound to bounded-loss generalization, let $\mathcal{B}$ be the class of sets $\{G : \ell_h(G) > s\}$ over all circuit coefficients and real thresholds s. Write P for the graph distribution and P_m for its empirical measure. The growth-function bound in Blumer et al. (1989, Proposition A2.1) gives at most $(2em/v)^v$ distinct traces of $\mathcal{B}$ on $2m$ graphs when $2m \geq v$. For a tolerance $\alpha > 0$ with $m\alpha^2 \geq 2$, an independent second sample reduces a population deviation to a deviation between the two empirical measures, with a factor of two in probability. Conditional on the paired samples, randomly swapping each pair makes each fixed trace a sum of independent bounded differences. Hoeffding's inequality and a union bound over the traces therefore give the conservative estimate

$$\Pr\left[\sup_{B \in \mathcal{B}} |P_m(B) - P(B)| > \alpha\right] \leq 8(2em/v)^v \exp(-m\alpha^2/32).$$

Finally, $\ell_h(G) = \int_0^1 \mathbf{1}\{\ell_h(G) > s\}\, ds$, so the same supremum bounds $\sup_h |P_m \ell_h - P\ell_h|$. This argument uses the fixed class of all candidate circuits, not a class conditioned on the learned features.

Taking $\alpha = \eta/2$ in this estimate and applying Hoeffding's inequality to the fixed reference algorithm's disagreement gives a universal constant c such that the additional sample requirement

$$m \geq \frac{c}{\eta^2}\left[v \log \frac{2}{\eta} + \log \frac{6}{\delta}\right] \tag{9}$$

ensures uniform convergence of the bounded losses and concentration of the reference algorithm's empirical disagreement fraction, each to tolerance $\eta/2$, with total failure probability at most $2\delta/3$. Together with the path event, this proves

$$\mathbb{E}\ell_{\widehat{h}} \leq \rho_{\text{end}} + \sqrt{n}\mathcal{A}_m\omega + \eta \tag{10}$$

with probability at least $1 - \delta$. Uniformity permits using the same graphs for feature construction and endpoint fitting.

To convert the loss bound into exact endpoint agreement, suppose a ReLU score $\widetilde{h}$ approximates $\widehat{h}$ uniformly within $1/12$. The clipped output with margin $\widehat{T}_u = \text{clip}_{[0,1]}(3\widetilde{h}_u - 1)$ equals every source label on any graph with $\ell_{\widehat{h}}(G) \leq 1/4$. For a zero label, this condition gives $\widetilde{h}_u \leq 1/4 + 1/12 = 1/3$; for a one label, it gives $\widetilde{h}_u \geq 3/4 - 1/12 = 2/3$. Markov's inequality applied to (10) yields

$$\text{err}_{\mathcal{D}}(\widehat{T}, T) \leq \min\{1,\, 4[\rho_{\text{end}} + \sqrt{n}\mathcal{A}_m\omega + \eta]\}. \tag{11}$$

This conversion explains the factor four on the reference algorithm's error in Theorem 2. It remains to construct the uniformly accurate ReLU score and account for numerical fitting error.

B.4 Numerical Computation and ReLU Compilation

We complete the proof of Theorem 2 by controlling finite-precision error and constructing the ReLU student. The regularizers control both the representation error and numerical conditioning. We give the coefficient and precision bounds because polynomial matrix dimensions alone do not establish a bit-time or network-size guarantee.

To bound the coefficients stored in one local update, fix an inner iteration and abbreviate its product matrix by $A = A_{t,k}$ and its fitted source-target matrix by $M = M_{t,k}$. The columns of M are centered because the normalized source targets are centered and $R_\mu(A)$ preserves the constant direction and its orthogonal complement. Let $A_\perp$ be the centered product matrix, and let $H_M := (M^*M + \theta^2 I)^{-1/2}$ be the retained-feature normalization matrix. The new nonconstant features have product coefficient matrix

$$\boldsymbol{\beta} = (A_\perp^* A_\perp + \mu^2 I)^{-1} A_\perp^* \overline{Q}_t^Z H_M, \qquad \|\boldsymbol{\beta}\|_{\mathrm{op}} \le \frac{1}{2\mu\theta}.$$

The ridge map $(A_\perp^* A_\perp + \mu^2 I)^{-1} A_\perp^*$ has operator norm at most $1/(2\mu)$; the remaining factors $\overline{Q}_t^Z$ and H_M have norms at most one and $1/\theta$. Let N_A be the maximum product-column count over all rounds and inner iterations. Every feature's coefficient ℓ_1 norm is at most $A_c := \sqrt{N_A}/(2\mu\theta)$. Centering adds an intercept of magnitude at most $A_c B_c^2$. The source regularizer does not enter this deployment bound because Q_t^Z is a contraction.

Finite-Precision Computation. We evaluate scalar operations in binary arithmetic, retaining enough working bits to control rounding error throughout the construction. For more background and expositions on rounding-error analysis, arbitrary-precision arithmetic, and the conditioning and computation of matrix functions, refer respectively to Higham (2002, Chapters 2–3), Brent and Zimmermann (2010, Chapter 3 and Section 4.3.3), and Higham (2008, Chapters 3, 6, and 8). We now use the positive offsets supplied by regularization to bound the precision needed by this learner. Every regularized Gram matrix has a positive eigenvalue floor. If $a_0 > 0$ is such a floor and $X, Y \succeq a_0 I$ are positive definite, their inverse and inverse square root satisfy the operator-norm bounds

$$\|X^{-1} - Y^{-1}\| \le a_0^{-2}\|X - Y\|, \qquad \|X^{-1/2} - Y^{-1/2}\| \le \tfrac{1}{2} a_0^{-3/2}\|X - Y\|.$$

The inverse bound follows from the resolvent identity. Integrating that identity in the representation $X^{-1/2} = \pi^{-1} \int_0^\infty t^{-1/2}(X + tI)^{-1}\, dt$, obtained from the standard square-root integral (Higham, 2008, Chapter 6, equation (6.1)), gives the second bound. These estimates control errors propagated between inner iterations using only the positive eigenvalue floor. We form Gram matrices symmetrically and keep their total operator-norm perturbation, including propagated input errors and rounding, below half that floor. The rounded matrices then remain positive definite, and the same estimates apply with the halved floor.

A concrete way to evaluate the matrix functions is the Cholesky and polar-decomposition construction of Higham (1986, Theorem 3.1 and Section 6.4). For a positive definite matrix $B = FF^\top$, the polar decomposition $F^\top = UH$ gives $H = B^{1/2}$; inversion then gives $B^{-1/2}$. The polar iteration replaces X by $(X + X^{-\top})/2$ and converges quadratically. If $a_0 I \preceq B \preceq b_0 I$, where b_0 bounds its largest eigenvalue, scaling before iteration bounds the required iteration count by a polynomial in $\log(b_0/a_0)$ and the requested accuracy in bits.

To control rounding over this finite iteration count, we reserve extra working bits, often called guard bits (Brent and Zimmermann, 2010, Section 4.3.3). After the first exact polar iteration,

every singular value is at least one. Keeping the accumulated perturbation below $1/2$ bounds all subsequent inverse norms by two. The inverse perturbation identity then bounds amplification in one polar iteration by $5/2$, so the additional guard bits beyond the scale-dependent input precision grow linearly with the number of iterations. Cholesky pivots and the first and final inversions are controlled by the same eigenvalue bounds; scalar square roots can be approximated by bisection. In the learner, the logarithms of these bounds are polynomial in the input encodings and regularizer encodings. Together with the preceding perturbation estimates and the polynomial number of inner iterations, this yields polynomial bit cost for any specified coefficient accuracy in bits.

More precisely, let b bound the bit lengths of the rational sample data and regularizers, and let $\zeta \in (0,1)$ be an additional loss tolerance. The computation takes bit time polynomial in the sample and architecture sizes, b, $\log^+(1/\sigma)$, $\log^+(1/\mu)$, $\log^+(1/\theta)$, and $\log(1/\zeta)$. Here $\log^+(x) = \max\{0, \log x\}$. The encoding length of each positive rational regularizer bounds its corresponding inverse logarithm, which yields the compact runtime statement in Theorem 2. This statement charges input encodings, including those of the regularizers. It excludes the cost of obtaining the source outputs; no finite-bit evaluation guarantee is assumed for an arbitrary real-valued source oracle.

The endpoint fit requires objective accuracy rather than continuity of the optimizing coefficients. Let ϵ_b bound the error in each final fitted feature value. Then the objective of every feasible coefficient vector changes by at most $R_0\epsilon_b$. Solving the perturbed LP to objective error ϵ_{LP} costs at most $2R_0\epsilon_b + \epsilon_{\mathrm{LP}}$ relative to the ideal features. The clipped local expressions also permit uniform control of deployment error from coefficient perturbations. Allocate these errors within ζ. The endpoint bound (11) then acquires an additional 4ζ before capping at one.

The same entrywise accuracy gives a computable upper bound on error amplification. Let $\widehat{Q}$ approximate a d-column ideal feature matrix Q entrywise within ϵ_b, and suppose $|Q_{(i,u),j}| \le B_c$ for every row and column. With weighted adjoints defined using the same Ω, we have

$$\|QQ^* - \widehat{Q}\widehat{Q}^*\|_{\infty\to\infty} \le d(2B_c\epsilon_b + \epsilon_b^2).$$

Indeed, the row weights sum to one. Add this allowance to the computed row-sum norm at each inner iteration, and substitute the resulting upper bounds into (8). Every coefficient of that expression is nonnegative, so the result certifies an upper bound on its ideal value.

Remark. We impose no lower bound on the nonzero singular values of the unregularized feature matrices and no spectral-gap assumption. The matrix computations above use the positive eigenvalue floors supplied by regularization; they do not require individual eigenvectors or eigenvalue separation.

Bounded Products Give a Polynomial ReLU Implementation. Each learned feature is a clipped affine combination of products of features bounded by B_c. Approximate every multiplication on $[-B_c, B_c]^2$ to error $\epsilon_\times$ using Yarotsky (2017, Proposition 3), and implement clipping exactly. For previous uniform error E, a product has error at most $2B_cE + \epsilon_\times$. The affine coefficient bound and nonexpansiveness of clipping control the next feature.

To accumulate these local approximation errors, let $H_f := r\sum_t D_t$ count learned feature nodes across all inner iterations and rounds, including coordinates not passed to the next round. Port routing is nonexpansive in uniform error, so a conservative bound for the accumulated feature error is

$$\epsilon_\times(1 + A_c)(1 + 2B_cA_c)^{H_f}.$$

Choose this bound to be at most $1/[12(1 + R_0)]$. The logarithm of the required multiplication precision is polynomial in H_f, $\log B_c$, $\log(1 + A_c)$, and $\log(1 + R_0)$. The product-slot count is polynomial as well. This proves polynomial compilation size in the architecture sizes, $\log(mn)$, and

the positive parts of the logarithms of inverse regularizers. Small numerical coefficient perturbations are covered by doubling the coefficient bound and reserving part of the same approximation budget. Together with the statistical bound in Appendix B.3, this completes the proof of Theorem 2.

Using the Computed Amplification Bound. For one round, $\mathcal{A}_m = Sr$. For several rounds, approximate readability alone does not ensure small error amplification. A threshold A_0 fixed before sampling can screen the computed amplification bound. Accept a fit only when its certified upper bound is at most A_0. The resulting error guarantee is $\min\{1, 4\rho_{\text{end}} + \epsilon\}$ on accepted runs when $\sqrt{n}A_0\omega + \eta + \zeta \le \epsilon/4$. Precisely, the probability that a fit is accepted and this guarantee fails is at most δ; the claim is not a conditional confidence statement given acceptance. A large amplification bound leaves this error guarantee uninformative; it does not establish failure of every learner.

Both the screening rule and the theorem concern endpoint error. The proof controls this error through linear combinations that approximate reference paths and states on the fitting graphs. It does not require each learned coordinate to recover a prescribed Boolean state. We next show that exact readability makes the endpoint bound small without screening the learned amplification factors.

B.5 Exact Readability Yields a Polynomial-Time Learner

We finish the proof of Theorem 1 by applying the regularized construction to exact path readability. Two estimates remove the extra premises of the approximate-readability guarantee. A rational source table gives a uniform bound on coefficients representing Boolean fitting vectors, and an elementary worst-case bound controls the empirical amplification. These bounds may be large, but their logarithms are polynomial. The numerical algorithm and ReLU compiler charge those logarithms.

The first estimate addresses poor conditioning: nearly dependent source features can require large coefficients to represent a Boolean path indicator. The determinant argument below shows that a rational table nevertheless admits a representation of every readable Boolean vector with coefficients bounded in terms of the table width and entry bit length. Combining this coefficient bound with the amplification estimate lets us choose regularization small enough to control bias across all rounds. The logarithm of the inverse regularization parameter remains polynomial, so the preceding numerical and ReLU-compilation estimates still give polynomial cost. This argument uses only a bound on the hidden readouts; the learner need not identify them.

Rational Tables Admit Bounded Fitting Readouts. We use the classical argument based on an invertible submatrix and Cramer's rule (Kaltofen et al., 2011, proof of Theorem 1), specialized here to Boolean vectors in the source column span. Let $d := \max_t(D_t + 1)$ and suppose that the numerator and denominator of each rational source entry, in reduced form, have absolute value at most 2^b, with denominators positive. The appended constant coordinate satisfies the same bound. Define

$$\overline{\tau} := d\, d!\, 2^{d^2 b}. \tag{12}$$

For any round, write $X = (1, Z^t)$ for its fitting table and let k be its rank. Choose k independent columns and k rows on which those columns form a nonsingular square matrix. Such rows exist by the definition of rank. The constant column ensures $k \ge 1$.

For any Boolean vector y in the column span of X, its unique coefficient vector in the chosen column basis is determined by those k rows. Clear denominators separately in each selected row. Each integer entry of the resulting square matrix and right-hand side has magnitude at most 2^{kb}: a row multiplier is the product of at most k denominators, and y is Boolean. The integer determinant is nonzero, hence has absolute value at least one. Cramer's rule bounds every coefficient numerator

by $k!2^{k^2b}$. After padding unused columns by zero, the coefficient norm is at most

$$\sqrt{k}\, k!2^{k^2b} \le \overline{\tau}.$$

The selected column basis spans the whole fitting table, so solving the selected rows also represents y on every other row. This single bound holds simultaneously for every Boolean y in the source column span. In particular, it holds for every path in the reference algorithm's tree on the fitting graphs. The algorithm need not know these vectors, select this column basis, or compute their coefficients. We use this argument only to bound the readout coefficients required by the analysis.

Exact population path readability implies zero empirical residual almost surely. The preceding bound therefore substitutes $\xi_m = 0$ and $\tau = \overline{\tau}$ in the deterministic recurrence (7). No population readout-norm bound, source-norm bound, or path-concentration sample requirement is needed in this specialization.

A Worst-Case Amplification Bound Suffices. For any vector f with $\|f\|_\infty \le 1$, its empirical norm is at most one. Since each $R_{t,k}$ is a contraction and each row weight is at least $1/(mn)$,

$$\|R_{t,k} f\|_\infty \le \sqrt{mn}.$$

Consequently $\Lambda_t \le \sqrt{mn}$. Together with $\chi_t \le \sqrt{n}$ and $m \ge 1$, this gives the explicit bound

$$\mathcal{A}_m \le \overline{A} := SrL(1 + 2Srnm)^{L-1}. \tag{13}$$

This bound need not be polynomial as a numerical value. Its logarithm is polynomial in the horizon and the logarithms of the displayed parameters, which is sufficient for choosing regularizers.

Fix the desired excess endpoint error $\epsilon \in (0, 1)$. Set $\eta = \zeta = \epsilon/16$, and choose the common ridge parameter $\sigma = \mu = \theta = \lambda$ to be the largest power of two not exceeding

$$\frac{\epsilon}{4n\overline{A}(\overline{\tau} + 3)}.$$

This choice is within a factor two of the displayed threshold. For a threshold $a \in (0, 1)$, choosing $\lambda = 2^{-\lceil \log_2(1/a) \rceil}$ gives $a/2 < \lambda \le a$ and uses $O(1 + \log(1/a))$ bits to encode its numerator and denominator. The deterministic state-error term obeys

$$\sqrt{n}\mathcal{A}_m \frac{\lambda}{2}(\overline{\tau} + 3) \le \frac{\epsilon}{8}.$$

Adding the statistical and numerical losses gives at most $\epsilon/4$. The endpoint conversion in (11) therefore proves

$$\mathrm{err}_{\mathcal{D}}(\widehat{T}, T) \le \min\{1, 4\rho_{\mathrm{end}} + \epsilon\}$$

with confidence at least $1 - \delta$. It suffices to meet the bounded-loss sample requirement (9). Its capacity parameters do not depend on m or the regularizer values, so this is a polynomial sample bound without screening fits by their amplification bounds.

Complexity and Scope. The logarithm of $\overline{\tau}$ is $O(d^2b + d\log(d + 1))$. Equation (13) also bounds $\log \overline{A}$ polynomially, so the chosen ridge parameter has polynomial encoding length. The finite-precision and compilation arguments in Appendix B.4 therefore give polynomial bit time and polynomial ReLU size in the sample and architecture sizes, the rational input bit length, and $\log(1/\epsilon)$. Obtaining the source evaluations is charged separately.

The rational-value premise concerns the activations actually supplied to the learner. Arbitrarily rounding exactly readable real-valued activations can destroy exact readability, so that operation is not justified by this theorem. Perturbed or approximately readable activations are instead covered by Theorem 2, with its explicit amplification term. Under exact readability, the result learns vector states and endpoint predictions; it does not assert recovery of the original Boolean intermediate readouts.

B.6 A Fast Graph-Sample Rate for Exact Endpoint Agreement

When the reference algorithm agrees exactly with the source, we can avoid estimating a general bounded loss. Tighter numerical tolerances make the fitted predictor satisfy the endpoint margins on every training graph. A consistent-learning bound then improves the dependence on the target error from quadratic to linear, up to logarithmic factors.

Corollary 4 (A fast graph-sample rate in the realizable case)**.** *Under the assumptions of Theorem 1, suppose $\rho_{\mathrm{end}} = 0$. Let v be the VC-dimension upper bound for the graph-loss level sets of the fixed circuit architecture used in Appendix B.3. For $\epsilon, \delta \in (0,1)$, it suffices to draw*

$$m \geq \max\left\{\frac{4}{\epsilon}\log_2\frac{2}{\delta}, \frac{8v}{\epsilon}\log_2\frac{13}{\epsilon}\right\}$$

independent graphs. The regularized construction, with the tolerances chosen below, returns a ReLU vector-state student with endpoint error at most ϵ and confidence at least $1-\delta$. Its bit runtime and network size retain the polynomial bounds of Theorem 1.

Proof. We first choose tolerances that enforce all fitting margins. We then apply a graph-level consistency bound and retain those margins through ReLU compilation.

Choose the sample size before observing the graphs. After obtaining the rational source tables, compute the uniform fitting-readout bound $\overline{\tau}$ from (12) and $\overline{A}$ from (13). Set the common ridge parameter to $\sigma = \mu = \theta = \lambda$, where λ is the largest power of two not exceeding

$$\frac{1}{16mn\overline{A}(\overline{\tau}+3)}.$$

Use a numerical empirical-loss allowance of at most $1/(32m)$. Here the allowance controls the raw empirical graph-maximum absolute loss of the stored circuit, including both the feature approximation and the approximate endpoint LP solution.

Exact path readability gives zero fitting residual almost surely. The deterministic recurrence and the regularizer choice imply

$$\sqrt{n}E_L \leq \sqrt{n\overline{A}}\,\frac{\lambda}{2}(\overline{\tau}+3) \leq \frac{1}{32m}.$$

Since $\rho_{\mathrm{end}} = 0$, the reference algorithm agrees with the source on every fitting graph almost surely. Its final regularized approximation is feasible in the bounded-coefficient endpoint LP. Consequently the returned circuit score h satisfies

$$\frac{1}{m}\sum_{i=1}^{m}\max_{u\in V(G_i)}|h_u(G_i) - T_u(G_i)| \leq \frac{1}{16m}.$$

Every summand is nonnegative, so every fitting graph has raw graph-maximum loss at most $1/16$.

To apply consistency, associate with each score h in the fixed circuit architecture its set of graphs that violate the output margin:

$$B_h = \left\{G : \max_{u\in V(G)} |\,\mathrm{clip}_{[0,1]}(h_u(G)) - T_u(G)| > 1/4\right\}.$$

This fixed-threshold class is contained in the graph-loss level-set class already bounded by v. The latter class contains the empty set, by taking the loss threshold above one. The fitted B_h contains

no training graph, so it is consistent with the empty target concept in that larger class. The consistent-learning theorem of Blumer et al. (1989, Theorem A2.2) therefore bounds its population probability by ϵ with confidence at least $1 - \delta$, at the displayed graph count. This argument is uniform over all circuit coefficients, so it allows the source-dependent regularizer, features, and endpoint coefficients to be selected using the same graphs. The clipping level mn is fixed once m is chosen; the VC bound does not depend on its magnitude.

Compile h into a ReLU score $\widetilde{h}$ with uniform error at most $1/12$. On every graph outside B_h, the clipped output with margin $\mathrm{clip}_{[0,1]}(3\widetilde{h}_u - 1)$ equals T_u at every vertex. This proves the endpoint guarantee without estimating a population loss or error of the reference algorithm from the fitting sample.

The stronger fitting tolerance changes the required precision by only $O(\log m)$ bits beyond the existing polynomial bounds: $\log \overline{\tau}$, $\log \overline{A}$, and the encoding length of the chosen ridge parameter are polynomial in the stated input parameters. The numerical allowance also has $O(\log m)$ encoding length. Thus the statistical improvement does not sacrifice the bit-time or ReLU-size guarantee. □

C Lower Bounds under Specified Source Access

The upper bounds show how readable paths support efficient distillation. This section proves the two limitations stated in Section 4.3: final predictions alone can leave a computational obstruction, and even exactly readable paths do not eliminate the need for sampled graphs. For the computational separation, we construct the same prediction task under two choices of source activations and verify exactly what each choice reveals. For the sample bounds, we construct targets that remain indistinguishable until the learner samples an informative graph.

C.1 A Computational Separation for the Same Student Class

The computational argument uses the pseudorandom-function framework of Goldreich et al. (1986, Sections 3–4). Its conclusion allows every efficiently evaluable predictor, following the representation-independent learning perspective of Kearns and Valiant (1994, Section 1). To apply this obstruction in our setting, we must realize the target with a small shared local decision tree while ensuring that the source's final predictions reveal no intermediate computation. The construction below establishes these properties with three incoming ports. We first specify the cryptographic assumption, the permitted source access, and the student class used on both sides of the comparison.

The key idea is that a secret key makes the local computation easy to execute when the key is known, while its final outputs remain hard to predict from output queries alone. An efficient learner that constructs such a predictor from these queries would distinguish these outputs from those of a random function, without needing to recover the key. For each fixed key, the two source interfaces have identical final predictions; the informative interface additionally exposes the path indicators needed by the positive distillation theorem.

Cryptographic Assumption. The construction uses a function that is easy to evaluate with a secret key but appears random to an efficient algorithm that only queries its values. Let κ be the *security parameter*: it specifies the key and input lengths, and we measure running time as a function of κ. Assume a family of Boolean pseudorandom functions $f_k : \{0,1\}^\kappa \to \{0,1\}$, indexed by keys $k \in \{0,1\}^\kappa$. A public algorithm evaluates $f_k(x)$ in polynomial time given (k, x).

The security assumption concerns an algorithm that does not know k. It may choose each query after seeing earlier answers. It tries to distinguish a uniformly keyed f_k from a random Boolean function, which assigns an independent fair bit to each input and repeats that bit on later queries to the same input. The algorithm outputs a bit; its *distinguishing advantage* is the absolute difference

between its probabilities of outputting one in these two cases. We assume that this advantage becomes smaller than every inverse polynomial in κ for every randomized polynomial-time algorithm, for all sufficiently large κ (Goldreich et al., 1986, Section 3.1). Such an algorithm is called *uniform*: one procedure handles all input lengths, rather than receiving separate advice for each length.

What the Distillation Learner Observes. A training query returns the full vector of source predictions and the designated node-indexed activations. For the lower bound, the learner may choose each query graph using earlier responses. It knows the graph distribution, communication pattern, initialization, and complexity bounds. It receives neither the key nor the source's parameters or code as separate inputs; these predictions and selected activations are its only source-specific observations. Its returned predictor must evaluate new graphs without calling the source. We count the bits needed to encode its parameters and the bit operations needed to run it, as in Theorem 1.

We compare two source interfaces, meaning two choices of which activations accompany the predictions. Both return the same raw attributes and final predictions in a common activation layout. Let $\mathcal{O}_k^-$ denote the interface that fills the remaining coordinates with zeros, and let $\mathcal{O}_k^+$ denote the interface that fills them with the reference tree's path indicators from Section 3.2. Their common activation dimension is D, and we use this same dimension at every round.

The Same Student Class on Both Sides. Both interfaces allow L-round message-passing students with at most $D+1$ coordinates passed between rounds, including a constant. Their local ReLU update modules share parameters across vertices but may differ across rounds. Each module has internal size and parameter bit length bounded by a polynomial in the problem and accuracy parameters. For each prescribed accuracy and confidence, choose one size and encoding budget that contains both the outputs of Theorem 1 and a direct implementation of the reference tree. Use the same budget for both interfaces; when accuracy and confidence are fixed, it is polynomial in κ. The negative result allows every efficiently evaluable predictor, so it includes this student class without requiring tree recovery or Boolean intermediate states.

With these access and size restrictions fixed, the next theorem makes Theorem 3 precise. Its positive and negative conclusions differ only in the designated activations.

Theorem 5 (Formal endpoint-access separation)**.** *Under the pseudorandom-function assumption above, there is a public family of graph distributions $\mathcal{D}_\kappa$ and paired source interfaces $\mathcal{O}_k^-$ and $\mathcal{O}_k^+$ with the same final predictions, initialization, communication pattern, activation dimensions and permitted student class, such that:*

1. *An unknown shared local decision tree exactly computes the final predictions in $L = \mathrm{poly}(\kappa)$ rounds on graphs with $n = \mathrm{poly}(\kappa)$ vertices. It has $S = O(\kappa)$ leaves and depth $r = O(\log \kappa)$. The graphs use three incoming ports and $a = O(\log \kappa)$ Boolean attributes, with direct concatenation of local inputs.*

2. *Under $\mathcal{O}_k^-$, no uniform probabilistic polynomial-time learner can guarantee endpoint error at most $1/4$ with probability at least $2/3$ for all keys at every sufficiently large κ. This holds even with polynomially many adaptive chosen-graph queries and arbitrary polynomial-time-evaluable output predictors.*

3. *Under $\mathcal{O}_k^+$, the supplied Boolean activations have dimension $D = O(\kappa)$ and exactly readable paths. Theorem 1 therefore returns a vector-state ReLU student with endpoint error at most ϵ, with probability at least $1-\delta$, in time and graph samples polynomial in $\kappa, 1/\epsilon, \log(1/\delta)$.*

The lower bound concerns learning from the specified observations: an exact ReLU student exists under both interfaces. The informative activations are constructed to satisfy exact path

readability. The positive result follows from Theorem 1, without ruling out simpler learners for this particular family.

Proof of Theorem 5. We encode a public evaluation circuit as a graph and use clock pulses to make only its output possibly nonzero at the final round. This ensures that the full vector of source predictions reveals only one pseudorandom-function value. We then supply either uninformative activations or exactly readable path indicators. Finally, an efficient learner using the former would distinguish a pseudorandom function from a random function by predicting fresh values.

A Public Circuit with Private Key Inputs. Let $E_\kappa(k, x)$ be a public polynomial-size Boolean circuit for $f_k(x)$, with fan-in at most two and gates AND, OR, NOT, COPY and constants. Unrolling the polynomial-time evaluator gives such a circuit in polynomial time by representing its computation with Boolean gates. The key bits are inputs to this public circuit; they are not encoded in graph attributes or edges. To implement this circuit one depth at a time, remove non-input gates outside the output's ancestry and insert COPY gates so that every wire joins consecutive circuit depths. Put input and constant vertices at depth one. Retain all κ public x-input vertices, including unused ones, so the resulting graph G_x uniquely records x. This will let the source validate queried graphs and recover x. Add a final COPY vertex $v_\star$ so it is the only data vertex at the maximum depth L. This uses polynomially many vertices and depths, even if the initial circuit is not shallow.

The graph G_x stores x_j as the raw bit at its corresponding input vertex. Key vertices store only their public index j, using the binary encoding of $j-1$ in $\lceil \log_2 \kappa \rceil$ bits. Every data vertex stores its gate role. These roles, ports, and indices are public and independent of k. Constants are input vertices whose raw bits are fixed. We also include distinct public vertex identifiers using $O(\log n) = O(\log \kappa)$ bits. They will allow the source to check whether a queried graph is a relabeling of this circuit graph by matching the identifiers. The distribution $\mathcal{D}_\kappa$ draws G_x by choosing x uniformly from $\{0,1\}^\kappa$; its graph structure and all attributes other than these input bits are fixed.

Clock Pulses Remove Intermediate Values from Final Predictions. A direct circuit simulation could leave key bits or intermediate gate values at other vertices in the final round. To prevent these values from appearing in the source's predictions, we activate each data vertex for exactly one round. Add clock vertices $c_0, \dots, c_{L-1}$. Initialize c_0 to one and every other vertex to zero. This is a known, key-independent initialization. Give c_0 the zero role, so it updates to zero. Each c_j, $j \geq 1$, has the clock role and copies the previous state of c_{j-1} through port one. Thus

$$z^t_{c_j} = \mathbf{1}\{t = j\}.$$

Stopping the chain at c_{L-1} makes every clock state zero at round L. Every data vertex of depth d receives c_{d-1} at port three. Its other two ports receive its circuit inputs, with missing inputs equal to zero. Write the previous states at these three ports as (y_1, y_2, c). The reference tree will implement a single shared local update, denoted by F_k, with the following rules:

role	next state
zero	0
clock	y_1
ordinary input	$c\,x_j$
key input of index j	$c\,k_j$
AND	$c(y_1 \wedge y_2)$
OR	$c(y_1 \vee y_2)$
NOT	$c(1 - y_1)$
COPY	cy_1

The ordinary-input value in this table is the vertex's raw bit; the only unknown part of the local function is the key lookup. The local input map directly concatenates the raw attributes, self state, and port states, as required by the upper bound.

At time d, a depth-d data vertex receives its correct circuit-input values and an active clock pulse. At all other times its clock input is zero. Induction on depth therefore gives its correct circuit value at time d and zero at every other time. In particular,

$$z^L_{v_\star}(G_x) = f_k(x), \qquad z^L_u(G_x) = 0 \quad (u \neq v_\star). \tag{14}$$

The second equality includes every clock vertex because the chain stops at c_{L-1}. Thus the full vector of final predictions contains only the output bit and fixed zeros.

It remains to check that the shared update has the required small decision tree. First query the three role bits. All non-key branches then use at most three additional bit tests. The key branch queries the clock bit, followed, when it is one, by the binary index; its leaves store the corresponding key bit. Pad invalid indices arbitrarily. A full index tree gives $S \leq 2\kappa + 21$ and depth at most $\lceil \log_2 \kappa \rceil + 6$ (loose bounds sufficient here). The resulting tree is the reference tree $F_\star$ for key k. Its depth grows logarithmically with κ, whereas the number of graph rounds follows the depth of the full evaluation circuit and may grow polynomially.

Realization in the ReLU Student Class. To make the comparison about learning, we also verify that the reference computation has an exact realization in the permitted student class. For a root-to-leaf path P, let $\ell_1, \ldots, \ell_j \in \{0, 1\}$ be the values of its $j \leq r$ literals on the local input. Its path indicator is

$$a_P = \mathrm{ReLU}(\ell_1 + \cdots + \ell_j - j + 1).$$

This expression equals one exactly when every literal is one and zero otherwise. Summing these indicators over leaves labeled one implements the tree with $O(Sr)$ local size. A student can store its Boolean state in one coordinate, retain a constant, and pad the remaining coordinates with zeros. Thus both source interfaces admit an exact target in the same vector-state ReLU class; the lower bound restricts learning from the observations, not representation of the target.

Final Predictions and Designated Activations. Set $T_u(G_x) = z^L_u(G_x)$ as in (14). The source accepts every relabeling of a well-formed G_x. Its public identifiers, ports, and designated input positions allow polynomial-time validation and recovery of x without knowing k. On every other graph, the source returns the all-zero prediction vector. This specifies the responses to chosen-graph queries outside the distribution support and prevents a learner from revealing a key bit by rewiring the circuit. The reference algorithm must reproduce the source only on $\mathcal{D}_\kappa$, as in Section 3.

Use a common activation layout consisting of the raw attributes, the final prediction $T_u(G)$, and $2S - 1$ additional coordinates. Under $\mathcal{O}^-_k$, all additional coordinates are zero. These activations add nothing beyond public graph data and the final predictions. In particular, the final reference state is linearly readable on the distribution support even under $\mathcal{O}^-_k$.

In $\mathcal{O}^+_k$, the coordinate indexed by path P at (t, u) contains

$$a_P\big(C_{G_x}(z^{t-1})_u\big),$$

for each root-to-node path P of the reference tree, with zero padding if needed. On malformed graphs these additional coordinates are zero in both interfaces. The source computes these quantities by executing its keyed local computation on the queried graph; the learner receives only the specified activations and final predictions. Each desired path indicator is one activation coordinate, so its readout has norm one. At most $r + 1$ path indicators are active at a vertex, so their vector has squared norm at most $r + 1$. All returned entries are Boolean and have constant bit length. The

graph size, number of rounds, tree size, and activation dimension are polynomial in κ, so Theorem 1 applies with $\rho_{\text{end}} = 0$. The source computes these indicators efficiently because it knows the key. An endpoint-only learner receives neither the key nor a procedure for computing these additional activations.

An Efficient Endpoint Learner Would Break Pseudorandomness. The remaining step proves the negative conclusion. Suppose a learner using $\mathcal{O}_k^-$ returns a polynomial-time predictor h that uses no source calls and has the claimed endpoint guarantee. Replace h by its coordinatewise threshold at $1/2$ if necessary; this cannot increase disagreement with Boolean target predictions. Whole-graph endpoint error at most $1/4$ implies that, with probability at least $2/3$,

$$\Pr_x \left[h(G_x)_{v_\star} = f_k(x)\right] \geq \frac{1}{2} + \alpha, \qquad \alpha = 1/4.$$

We use this prediction advantage to build a distinguisher with query access to a challenge function g, which is either f_K for a uniform key K or a random Boolean function. The distinguisher simulates every well-formed learner query by one query to g, returning $g(x)$ at $v_\star$ and zeros at all other vertices, together with the activations of $\mathcal{O}_k^-$. Malformed queries need no call to g. Impose the learner's promised polynomial runtime bound, returning an arbitrary predictor on timeout; this makes the simulation bounded even on a random-function oracle, which need not satisfy the realizability promise. After training, it draws $M = \lceil 2\alpha^{-2} \log 24 \rceil$ fresh uniform inputs. It outputs one if the predictor agrees with g on at least a $1/2 + \alpha/2$ fraction of these validation inputs. The predictor makes no further calls to g. Apply the promised polynomial evaluation time limit to each predictor evaluation and return an arbitrary bit on timeout or malformed output. This bounds the distinguisher even when the random-function transcript produces invalid predictor code.

If $g = f_K$, the simulation is exactly the specified training interface. The learner's guarantee and Hoeffding's inequality therefore give acceptance probability at least $(2/3)(23/24)$. If g is a random function, condition on the complete training transcript and returned predictor. Except when a validation input repeats a training query or another validation input, the corresponding function values are fresh independent fair bits. If training made at most Q oracle queries, the exceptional probability is at most

$$\frac{QM + \binom{M}{2}}{2^\kappa}.$$

Hoeffding's inequality therefore bounds acceptance in the random case by $1/24 + (QM + \binom{M}{2})2^{-\kappa}$. Since Q and M are polynomially bounded, the collision term tends to zero. The two acceptance bounds give a constant distinguishing advantage, contradicting the assumed pseudorandom-function security. This contradiction uses only efficient evaluation of h, so it proves the negative conclusion for every permitted predictor and completes the separation. □

C.2 Unconditional Requirements on the Number of Sampled Graphs

The computational separation concerns what an efficient learner can infer from its observations. A different limitation remains even with unlimited computation: independently sampled graphs may fail to reveal the labels needed for accurate prediction. We adapt the classical unseen-label argument of Ehrenfeucht et al. (1989, Theorems 1 and 4) to graph computations with exactly readable paths. These bounds allow arbitrary output predictors, including the ReLU students of the main theorems.

For both bounds, access consists of independent graph samples together with their designated source activations and final predictions. The learner receives neither chosen-graph queries nor source parameters. The first construction isolates the dependence on accuracy and confidence by placing the only unknown label on a rare graph.

Proposition 6 (Accuracy and confidence require sampled graphs)**.** *For $0 < \epsilon \le 1/4$ and $0 < \delta < 1/2$, there is a family with exactly readable paths, one vertex, one round, one Boolean input attribute, activation dimension one, and a reference algorithm whose tree has at most two leaves such that every learner using m sampled graphs has, for some target, probability at least*

$$\frac{1}{2}(1-2\epsilon)^m$$

of returning endpoint error greater than ϵ. Thus an (ϵ, δ) guarantee for every target requires

$$m \ge \frac{\log(1/(2\delta))}{-\log(1-2\epsilon)} \ge \frac{1}{4\epsilon}\log\frac{1}{2\delta}.$$

The learner's computation and output class are unrestricted.

Proof. We choose two targets that differ only on a graph of probability 2ϵ. Until that graph is sampled, their source responses are identical, so no learner can reliably distinguish them.

Let G_x have a single vertex with raw bit x, known initial state zero, and one update round. Put $\Pr[x = 1] = 2\epsilon$. The two possible teachers are

$$T_\theta(G_x) = \theta x, \qquad \theta \in \{0, 1\},$$

and both expose the same source feature $Z(G_x) = x$. A tree querying x, with leaf labels zero and θ, implements an exact reference algorithm. Its path indicators $1, x, 1-x$ are linearly readable from $(1, Z)$ with constant-size rational coefficients. Thus both teachers satisfy exact path readability with the same known initialization and communication pattern.

With probability $(1-2\epsilon)^m$, no sampled graph has $x = 1$. Conditional on this event, both targets produce identical observed inputs, source activations, and final predictions. For any fixed transcript and learner random seed, the returned prediction at G_1 cannot equal both zero and one. Averaging over a uniform choice of θ therefore gives probability at least one half of an incorrect prediction there. Such a prediction incurs graph error at least $2\epsilon > \epsilon$. Averaging over the two targets gives the stated failure probability for at least one fixed target. The necessary sample bound follows by rearranging; the final inequality uses $-\log(1-2\epsilon) \le 4\epsilon$. □

The restriction to independent samples is essential: a chosen-graph query at G_1 would immediately reveal θ. Under independent sampling, the proposition shows why the accuracy and confidence factors in Corollary 4 persist even when the update requires only two leaves.

The next construction adds many rare graphs with independently varying labels. Its activation coordinates identify these graphs but do not reveal an unseen label. This gives a sample requirement that grows with the source activation dimension.

Proposition 7 (An activation-dimension sample requirement)**.** *For every integer $d \ge 1$ and $0 < \epsilon \le 1/16$, there is a family with exactly readable paths, one vertex, one round, activation dimension $D = d$, $a = \lceil \log_2(d+1) \rceil$ raw Boolean attributes, and reference algorithms whose trees have fewer than $2(d+1)$ leaves and depth a, such that every learner using $m \le d/(32\epsilon)$ sampled graphs has, for some target, probability at least $1/16$ of returning endpoint error greater than ϵ. The learner's output class and computation are unrestricted.*

Proof. We distribute a total probability mass 8ϵ over d graphs and choose their target labels independently. With the stated number of samples, many of these labels remain unobserved. We then convert the uncertainty about these labels into a probability of endpoint error greater than ϵ.

Let G_j, $0 \le j \le d$, be a single-vertex graph whose raw bits encode its index, with known initial state zero and one update round. Give G_0 probability $1 - 8\epsilon$ and each other graph probability $8\epsilon/d$. Targets are indexed by $\theta \in \{0,1\}^d$, with $T_\theta(G_0) = 0$ and $T_\theta(G_j) = \theta_j$ for $j \ge 1$. All targets expose $Z(G_0) = 0$ and $Z(G_j) = e_j$ for $j \ge 1$, where $e_j \in \mathbb{R}^d$ is the jth coordinate vector. A complete depth-a index tree realizes every target, with fixed arbitrary labels on unused codes. To verify readability, a path that excludes index zero has indicator equal to the sum of the corresponding nonzero index coordinates. A path containing index zero has indicator equal to one minus the sum of the excluded nonzero index coordinates. Each path is therefore exactly readable from $(1, Z)$ on the distribution support. All source entries are zero or one, and the reference algorithm matches the teacher exactly.

Initially choose the target bits independently and uniformly. Let N count sampled nonzero indices, including repetitions. Since $\mathbb{E}N = 8\epsilon m \le d/4$, Markov's inequality gives $\Pr[N \le d/2] \ge 1/2$. On this event, let k be the number of nonzero indices that were never observed; then $k \ge d/2$. Conditional on the transcript and learner random seed, their labels are still independent fair bits. The number M of errors on them stochastically dominates $B \sim \mathrm{Binomial}(k, 1/2)$; non-Boolean predictions only increase the number of exact endpoint disagreements.

To lower-bound the probability of sufficiently many mistakes, apply the Paley–Zygmund inequality, which follows from Cauchy–Schwarz, to B. Since $\mathbb{E}B = k/2$ and $\mathbb{E}B^2 = k(k+1)/4$, it gives

$$\Pr[B > k/4] \ge \frac{1}{4}\frac{(\mathbb{E}B)^2}{\mathbb{E}B^2} = \frac{k}{4(k+1)} \ge \frac{1}{8}.$$

Since $k \ge d/2$, the event $M > k/4$ implies $M > d/8$, so the endpoint error exceeds $(8\epsilon/d)(d/8) = \epsilon$. Combining the events gives failure probability at least $1/16$, averaged over targets, and hence for some fixed target. □

The two propositions establish accuracy, confidence, and activation-dimension dependence for independent graph samples, even under exact path readability. In the second construction, tree size also grows with the activation dimension. These bounds do not match every architecture factor in the upper bounds or establish that readable paths are necessary. Together with the computational separation, they distinguish the resources studied in Section 4.3: appropriate activations can remove a computational obstruction, while observing informative graphs remains necessary. The next section specifies how the experiments evaluate the representation conditions and prediction error in finite samples.

D Experimental Details

Section 5 uses two constructed sources to examine learning under exact path readability and error accumulation under approximate path readability. We first specify their common learning and evaluation protocol, then describe each source, its readability checks, and the corresponding results.

D.1 Learning Protocol and Numerical Evaluation

To separate the representation assumption from the outcome of learning, we check each source's readability independently of fitting. The learner receives graph attributes, known initialization, labeled incoming ports, source activations, and source endpoint predictions. It receives no reference intermediate states, decision tree, path labels, or path readouts. Reference trajectories are used to define the sources and verify their readability. Source and reference endpoints agree, so $\rho_{\mathrm{end}} = 0$ in both experiments.

We implement the regularized construction of Section 4. Each round collects a constant, attributes, and previous learned self/port vectors into U^t, starting from $(1, z^0)$. This matrix stays fixed throughout the r inner iterations. Starting from $Q_{t,0} = \mathbf{1}$, each iteration forms every entrywise product of a column of U^t and a column of $Q_{t,k-1}$, regresses toward the round's centered normalized source features, and normalizes the fitted features. Constants are retained without regularization. With a common ridge parameter $\lambda > 0$, we set $\sigma = \mu = \theta = \lambda$: the normalization covariances use offsets $\lambda^2 I$, and ridge regression uses the squared coefficient penalty λ^2. We clip nonconstant coordinates to $[-mn, mn]$. A source of dimension D gives $D + 1$ retained coordinates. The final linear program fits the readout coefficient vector γ by minimizing the average, over fitting graphs, of the largest absolute discrepancy between the unclipped score and the source prediction at any vertex, subject to $\|\gamma\|_1 \le \lceil \sqrt{D+1} \rceil$, as in (6). Stored affine product maps permit evaluation without source access.

To evaluate this fit using the same error criterion as the main paper, let $h_u(G)$ denote its score at vertex u. The output applies clipping with a margin, while the auxiliary loss uses ordinary clipping:

$$\widehat{T}_u = \mathrm{clip}_{[0,1]}(3h_u - 1), \qquad \ell_h(G) = \max_u |\,\mathrm{clip}_{[0,1]}(h_u) - T_u|.$$

Numerical graph error means that some vertex's deployed output differs from its Boolean target by more than 10^{-8}. Correct classifications at threshold $h_u = 1/2$ can still fail this criterion: for example, a score of 0.6 has label one at that threshold but clipped output 0.8. Independent graphs are the sampling units; nodes and rounds are dependent. Constant graph sizes make row averaging equivalent to equal weighting of graphs and vertices within graphs. Reported m counts fitting graphs; validation and test access are charged separately below.

Fitting uses float64 arithmetic products in both experiments. The parity curves evaluate the stored arithmetic construction. The finite-population experiment additionally replaces products by ReLU multiplication approximations built from 24 iterations of a piecewise-linear tent map, using the square-approximation construction of Yarotsky (2017, Propositions 2–3). We evaluate the resulting student on every graph in the finite population. Linear algebra uses NumPy/SciPy and endpoint programs use HiGHS. These numerical implementations provide a proof of concept. They do not establish the theorems' universal bit-precision schedule, asymptotic sample rates, or guarantees for arbitrary learned sources.

D.2 Regularized Learning with Exactly Readable Parity Paths

The first experiment tests whether the learner can construct useful local features when every reference-tree path is exactly readable. Mixing the path indicators with nuisance coordinates supplies this information without giving the learner the individual indicators or their readouts.

Distribution and source. Each graph has eight vertices, independent fair Boolean attributes x_u and initialization bits z_u^0, and one incoming port per vertex. Write $\pi(u)$ for the sender at vertex u's incoming port. These senders are independent uniform vertices and may include u itself. With $\oplus$ denoting Boolean exclusive-or, the shared update is

$$z_u^t = x_u \oplus z_u^{t-1} \oplus z_{\pi(u)}^{t-1}.$$

Its complete tree tests these three bits in order, with $S = 8, r = 3$ and a vector $p_u^t \in \{0,1\}^{14}$ of nonempty-path indicators. To mix these indicators with additional information, let $d_u \in \{0,1\}^4$ contain the graph-wide parities of z^0 and x, and $z_{u-3}^0 \oplus x_{u-2}$ and $z_{u-4}^0 \oplus x_{u-5}$, with indices modulo eight. Let $D_\pm$ be a diagonal sign matrix and P a permutation matrix, both of size 18 and fixed with

seed 79319. Define the mixing matrix M below and treat the feature vectors in this construction as rows. The source returns

$$Z_u^t = (p_u^t, d_u)M, \qquad M = (I_{18} + \mathbf{1}\mathbf{1}^\top)D_\pm P.$$

The mixing matrix is hidden from the learner. It has condition number 19 and determinant -19; columns of its rational inverse recover every nonempty path indicator, while the root indicator uses the appended constant. Exact checks cover all eight local contexts and sixteen nuisance configurations. Thus the source satisfies the representation premise of Theorem 1 independently of the sampled graphs. Fitting uses $D = 18$, nineteen retained coordinates, and endpoint coefficient bound five.

Selection and confirmation. Having verified the source assumption, we choose the regularizer in a pilot and then measure learning on fresh graphs. The 27 pilot fits cross $m \in \{16, 64, 256\}$, $L \in \{1, 2, 3\}$, and $\lambda \in \{10^{-2}, 10^{-3}, 10^{-4}\}$, with one fitting seed. The pilot uses 336 fitting draws across the nonnested budgets, reused across horizons and regularizers, plus 1024 shared validation graphs. Selection minimizes mean validation graph error across the nine combinations of fitting budget and horizon, breaking ties by mean validation loss and then larger regularization. This selects $\lambda = 10^{-2}$. The feature construction and endpoint LP follow Section 4; the regularizer is chosen by this pilot, rather than by the conservative accuracy-dependent schedule in Appendix B.5. Pilot training and validation generators use seeds 350000 and 551991.

After this selection, the confirmation uses 27 fits: three budgets, three horizons, and fresh seeds $301, 302, 303$, with training/test generators $750000 + s$ and $951991 + s$. Each seed has 2048 fresh test graphs, reused across its budget/horizon settings. No confirmation result enters selection. Figure 3 includes every confirmation setting; the archive contains the individual error counts. All saved models replay identically with source/reference calls disabled. These are numerical learning results under an exactly readable source, with a separately charged pilot, rather than measurements of the theorem's sample exponent.

D.3 Exhaustive Finite-Population Check of the Approximate Bound

The second experiment introduces a nonzero path-readout error and checks how the learned reconstruction factors amplify it. A population of only 64 graphs lets us evaluate both prediction error and the loss gap over every possible input, subject to floating-point arithmetic.

Distribution and approximate source. The population is uniform on all 64 assignments of three gate bits g_u and three initialization bits on a directed three-cycle, with incoming port $\pi(u) = u + 1 \pmod 3$. The shared multiplexer update is

$$z_u^t = g_u z_u^{t-1} + (1 - g_u) z_{\pi(u)}^{t-1}.$$

Its gate-then-selected-state tree has $S = 4, r = 2$. Writing $s = z_u^{t-1}$ and $b = z_{\pi(u)}^{t-1}$, its six nonempty-path indicators form the vector $p_u^t = (g_u, 1 - g_u, g_u s, g_u(1 - s), (1 - g_u)b, (1 - g_u)(1 - b))$. To perturb these indicators before mixing, append two zeros to form $\bar{p}_u^t \in \mathbb{R}^8$. Let H_8 be the Sylvester Hadamard matrix of order eight and $\varepsilon_u^t \in \{-1, 1\}^8$ a deterministic sign vector. The source returns the row vector

$$Z_u^t = (\bar{p}_u^t + \xi\varepsilon_u^t)H_8/4, \qquad H_8 H_8^\top = 8I.$$

The signs are SHAKE-256 functions of the serialized graph, round, and node position, fixed before sampling; repeated graphs receive identical source values. This node-indexed source need not be relabeling-equivariant. To check approximate readability, let e_j be the j-th standard basis vector

in $\mathbb{R}^8$. For nonempty path j, the augmented readout $w_j = (0, H_8^\top e_j/2)$ has norm $\sqrt{2}$ and satisfies $(1, Z_u^t)w_j = p_{u,j}^t + \xi\varepsilon_{u,j}^t$. Its pointwise absolute residual is therefore exactly ξ. The root indicator uses $w_{\text{root}} = (1, 0)$, with norm one and zero residual. These readouts are shared across graphs and vertices and fixed independently of fitting. Since two path entries are active,

$$\|(1, Z_u^t)\|_2 \le \sqrt{2 + 2\xi + 4\xi^2}, \qquad \tau = 2.$$

These specified readouts have nonzero residuals; we also check that a different readout cannot recover a nonempty path exactly. The source matrix over all population vertices has exact rational rank nine after appending the constant and ten after adjoining any nonempty path, at every tested round and noise level. Hence no exact affine readout of a nonempty path exists on this distribution, so these paths have strictly positive optimal residuals. The tested perturbations and mixing matrix have rational entries whose denominators are powers of two, meeting the source-encoding requirement in Theorem 2.

With this source fixed, the 54 runs cross $m \in \{64, 256\}$, $L \in \{1, 2, 3\}$, $\xi \in \{2^{-20}, 2^{-16}, 2^{-12}\}$, and seeds $s = 0, 1, 2$. All use $\lambda = 2^{-20}$ without validation selection, source dimension $D = 8$, nine retained coordinates, and endpoint coefficient bound three. Training draws are independent samples with replacement, using generator $361000 + s$. The $m = 64$ samples contain $39, 42, 43$ distinct support graphs; $m = 256$ contains $62, 62, 61$. Arithmetic and ReLU evaluations enumerate all 64 configurations, including those absent from fitting.

Measured allowance and outcomes. To connect these evaluations to the approximate theorem, we measure the arithmetic student's generalization gap and the score change caused by replacing its products with ReLU approximations. Let $\bar{\ell}_h$ and $\widehat{\ell}_h$ denote the population and fitting means of the auxiliary loss, and let $\widetilde{h}$ denote the score after replacing arithmetic products by ReLU approximations. Exhaustive evaluation gives

$$\Delta_{\text{gap}} = \max\{0, \bar{\ell}_h - \widehat{\ell}_h\}, \qquad \Delta_{\text{ReLU}} = \max_{G \in \text{supp}(\mathcal{D}), u} |\widetilde{h}_u(G) - h_u(G)|.$$

The remaining term tracks the representation error using the recurrence in Appendix B.3. The fitting matrices determine the accumulated amplification bound $\mathcal{A}_m$; cycle routing has $\chi_t = 1$. The maximum empirical path residual is $\xi_m = \xi$ pointwise, so we use the deterministic recurrence directly with $\omega_m = \xi + \mu + \frac{1}{2}(\sigma\tau + \theta)$. This bypasses concentration for these readouts; it is not an invocation of Theorem 2 with $\nu = 0$. Substituting $\tau = 2$ gives the checked quantity

$$B_{\text{measured}} = \min\left\{1,\ 4\left[\sqrt{3}\,\mathcal{A}_m(\xi + 5 \cdot 2^{-21}) + \Delta_{\text{gap}} + \Delta_{\text{ReLU}}\right]\right\}.$$

All recorded recurrence and endpoint-fit comparisons pass with additive tolerance 10^{-7}, and exhaustive ReLU graph error is at most B_{measured}. This bound uses the measured loss gap and the maximum error from replacing products with the fitted coefficients fixed.

The measured allowance excludes numerical errors in fitting and solving the linear program, and does not certify the amplification factors. Thus this is a numerical check of the error accumulation underlying Theorem 2 on a known distribution, rather than a verification of its sample condition or training-confidence guarantee. The arithmetic is not interval-certified. Table 2 gives the results for every tested budget, horizon, and residual, including the smaller-budget failures mentioned in the main paper.

Table 2 reports the maximum error and uncapped bound range across seeds for every condition. Bounds below one are informative; larger values become vacuous after capping. Increasing the horizon and residual loosens the bound, even when predictions stay accurate. All failures occur

		$m = 64$		$m = 256$	
L	ξ	Max. error (%)	Uncapped bound range	Max. error (%)	Uncapped bound range
1	2^{-20}	0	0.000185	0	0.000185
1	2^{-16}	0	0.000978	0	0.000978
1	2^{-12}	0	0.013664	0	0.013664
2	2^{-20}	0	0.00816–0.00917	0	0.00778–0.00830
2	2^{-16}	0	0.04313–0.04847	0	0.04114–0.04385
2	2^{-12}	0	0.60270–0.67730	0	0.57487–0.61265
3	2^{-20}	9.375	0.49876–1.03588	0	0.38650–0.47591
3	2^{-16}	9.375	2.49084–3.83418	0	1.94515–2.49491
3	2^{-12}	9.375	34.7955–49.1359	0	27.1350–34.8895

Table 2: Approximate-path results over all 64 population graphs. Each error is the largest ReLU graph error over three seeds; each bound range is the minimum and maximum before capping at one.

at the smaller fitting budget and longest horizon, including some cases with correct threshold classifications. At $m = 256$, all tested conditions have zero numerical graph error, although some bounds remain vacuous.